SIMUNET: API LOAD TESTING AND CHAOS-INJECTION DASHBOARD

A MINI PROJECT REPORT

Submitted by

STEPHIN SIBY

MLM25MCA-2052

To

the APJ Abdul Kalam Technological University in
Partial fulfillment of the requirements for the
award of the degree

of

MASTER OF COMPUTER APPLICATIONS

Under the Guidance of

Ms. Athira R Kurup

(Assistant Professor, Department of Computer Applications)

DEPARTMENT OF COMPUTER APPLICATIONS

MANGALAM COLLEGE OF ENGINEERING, ETTUMANOOR

(Affiliated to APJ Abdul Kalam Technological University)

OCTOBER 2026

VISION

To become a centre of excellence in computer applications,competent in the global ecosystem with technical knowledge,innovation with a sense of social commitment.

MISSION

- To serve with state of the art education,foster advanced research and cultivate innovation in the field of computer applications.
- To prepare learners with knowledge skills and critical thinking to excel in the technological landscape and contribute positively to society.

Program Educational Objectives

- PEO I :Graduates will possess a solid foundation and in-depth understanding of computer applications and will be equipped to analyze real-world problems, design and create innovative solutions, and effectively manage and maintain these solutions in their professional careers.
- PEO II: Graduates will acquire technological advancements through continued education, lifelong learning and research, thereby making meaningful contributions to the field of computing.
- PEO III: Graduates will cultivate team spirit, leadership, communication skills, ethics, and social values, enabling them to apply their understanding of the societal impacts of computer applications effectively.

Program Specific Outcomes

- **PSO I**: Apply advanced technologies through innovations to enhance the efficiency of design and development.
- **PSO II**: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

MAPPING OF PO-PSO-SDG

1. MAPPING WITH PROGRAM OUTCOMES (POs):

SL.NO	POs ADDRESSED	RELEVANCE TO PROJECT
1	PO1	Applies programming, networking, asynchronous execution, and measurement concepts.
2	PO2	Analyses the need to observe API behaviour under concurrent load.
3	PO3	Designs a dashboard and backend workflow for load generation and telemetry.
4	PO4	Uses modern web development and monitoring libraries.
5	PO5	Plans, documents, implements, and presents a software project.
6	PO6	Uses a staged mini-project plan and mostly open-source components.
7	PO7	Emphasises ethical use against authorised test targets.
8	PO8	Builds familiarity with current web and performance engineering tools.

LIST OF PROGRAM OUTCOMES (POs):

PO1 (**Foundation Knowledge**): Apply knowledge of programming logic, database concepts and coding fundamentals for solution architecture and problem solving.

PO2 (**Problem Analysis**): Identify, review, formulate and analyze problems in traditional complaint reporting with focus on user requirements.

PO3 (**Development of Solutions**): Design and develop an innovative online complaint reporting solution for efficient complaint management and tracking.

PO4 (**Modern Tool Usage**): Select and apply modern computational tools and technologies such as PHP, MySQL, JavaScript and NLP for system development.

PO5 (**Individual and Teamwork**): Function and communicate effectively as an individual or team member during the planning, development and testing of the project.

PO6 (**Project Management and Finance**): Apply project management principles such as planning, scheduling, resource management and work breakdown structure.

PO7 (**Ethics**): Follow professional ethics and cybersecurity practices to protect user information, complaint data and digital evidence from unauthorized access.

PO8 (Life-long learning): Develop change management skills and continuously learn and adapt to contemporary technologies and methods for improving the complaint reporting system.

2. MAPPING WITH PROGRAM SPECIFIC OUTCOMES (PSOs):

LIST OF PROGRAM SPECIFIC OUTCOMES (PSOs):

SL.NO	PSOs ADDRESSED	RELEVANCE TO PROJECT
1	PSO1 – Apply advanced technologies through innovations to enhance the efficiency of design development	Advanced technologies such as WebRTC, WebSockets.
2	PSO2 – Apply the principles of computing to analyze, design and implement sustainable solutions for real-world challenges	Computing principles of data modelling, networking, security and software design were applied to build a sustainable solution for remote technical hiring.

PSO 1: Apply advanced technologies through innovations to enhance the efficiency of design development.

PSO 2: Apply the principles of computing to analyze, design and implement sustainable solutions for real world challenges.

3. MAPPING WITH SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG NO	SDGs ADDRESSED	RELEVANCE TO PROJECT
SDG 4	Quality Education	Through an educational software engineering tool, and an indirect relationship
SDG 9	Industry, Innovation and Infrastructure	Through the study of digital service reliability. These links are qualitative and should not be presented as measured impact.

SUSTAINABLE DEVELOPMENT GOALS (SDGs):

SDG 1 – No Poverty: End poverty in all its forms everywhere.

SDG 2 – Zero Hunger: End hunger, achieve food security and improved nutrition, and promote sustainable agriculture.

SDG 3 – Good Health and Well-Being: Ensure healthy lives and promote well-being for all at all ages.

SDG 4 – Quality Education: Ensure inclusive and equitable quality education and promote lifelong learning opportunities for all.

SDG 5 – Gender Equality: Achieve gender equality and empower all women and girls.

SDG 6 – Clean Water and Sanitation: Ensure availability and sustainable management of water and sanitation for all.

SDG 7 – Affordable and Clean Energy: Ensure access to affordable, reliable, sustainable and modern energy for all.

SDG 8 – Decent Work and Economic Growth: Promote sustained, inclusive and sustainable economic growth, full and productive employment and decent work for all.

SDG 9 – Industry, Innovation and Infrastructure: Build resilient infrastructure, promote inclusive and sustainable industrialisation and foster innovation.

SDG 10 – Reduced Inequalities: Reduce inequality within and among countries.

SDG 11 – Sustainable Cities and Communities: Make cities and human settlements inclusive, safe, resilient and sustainable.

SDG 12 – Responsible Consumption and Production: Ensure sustainable consumption and production patterns.

SDG 13 – Climate Action: Take urgent action to combat climate change and its impacts.

SDG 14 – Life Below Water: Conserve and sustainably use the oceans, seas and marine resources.

SDG 15 – Life on Land: Protect, restore and promote sustainable use of terrestrial ecosystems, manage forests sustainably, combat desertification and halt biodiversity loss.

SDG 16 – Peace, Justice and Strong Institutions: Promote peaceful and inclusive societies, provide access to justice for all and build effective, accountable and inclusive institutions.

SDG 17 – Partnerships for the Goals: Strengthen the means of implementation and revitalise the global partnership for sustainable development.

MANGALAM COLLEGE OF ENGINEERING,

ETTUMANOOR DEPARTMENT OF COMPUTER APPLICATION



DECLARATION

I hereby certify that the work presented in the project entitled "SIMUNET-API LOAD TESTING AND CHAOS INJECTION DASHBORD" submitted to the DEPARTMENT OF COMPUTER APPLICATIONS is an authentic record of my own work carried out under the supervision of Ms. Athira R kurup, Assistant Professor. This work has not been submitted to any other institution or university for the award of any other degree. The report has been checked for plagiarism by the college and the similarity index is within the permissible limit set by the college.

Name & Signature of Student

STEPHIN SIBY

Date:

Place:

MANGALAM COLLEGE OF ENGINEERING,

ETTUMANOOR DEPARTMENT OF COMPUTER APPLICATION



CERTIFICATE

*Certified that the report entitled "**SIMUNET: API LOAD TESTING AND CHAOS-INJECTION DASHBOARD**" submitted by **STEPHIN SIBY (MLM25MCA-2052)** to the **APJ Abdul Kalam Technological University** in partial fulfillment of the requirements for the award of **the Degree of Master of Computer Applications** is a bonafide record of the MINI PROJECT work carried out by the student under our guidance and supervision during the academic year 2025-27.*

PROJECT GUIDE

Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

PROJECT COORDINATOR

Ms. Athira R Kurup
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT

Ms. Divya S B
Department of Computer Applications

INTERNAL EXAMINER

ACKNOWLEDGEMENT

I am greatly indebted to the authorities of Mangalam College of Engineering for providing the necessary facilities to successfully complete my Project on the topic **"*SIMUNET: API LOAD TESTING AND CHAOS-INJECTION DASHBOARD*".**

I express my sincere thanks to **Dr. Vinodh P Vijayan**, Principal, Mangalam College of Engineering, for providing the facilities and support to complete my Project successfully.

I thank and express my sincere gratitude to **Ms. Divya S B**, HOD, Department of Computer Applications, Mangalam College of Engineering, for her invaluable help and support, which helped me a lot in successfully completing this Project work.

I express my gratitude to my Internal Guide, **Ms. Athira R Kurup**, Assistant Professor, Department of Computer Applications, for her valuable suggestions, guidance, and encouragement, which helped in the successful completion of my Project.

Furthermore, I would like to acknowledge with much appreciation the crucial role of the faculties, especially Project Coordinator, **Ms. Athira R Kurup**, Department of Computer Applications, Mangalam College of Engineering, who gave me the necessary guidance, permission, and resources required to complete the project presentation and report.

Finally, I would like to express my heartfelt thanks to my parents for their continuous support, encouragement, and motivation throughout the completion of this Project.

STEPHIN SIBY

MLM25MCA-2052

ABSTRACT

SimuNet is a browser-based prototype for API load testing and basic chaos-style delay injection. It combines a React and Vite frontend with a Node.js and Express backend. The backend creates a worker thread that issues concurrent HTTP GET requests through Axios, optionally waits for a configurable artificial delay before each request, and reports interval metrics to the main process. Socket.IO is used to broadcast live metrics to the dashboard, where chart components can visualise changing request behaviour. A stop operation returns a summary containing duration, total requests, peak interval count, average latency for successful requests, and error rate. The project demonstrates the integration of web UI design, asynchronous request generation, worker threads, event-driven telemetry, and basic performance measurement. The current implementation is a learning-oriented prototype and has limitations in input validation, request timeouts, session isolation, metric precision, and persistent storage. Features mentioned in planning slides, such as Redis-backed storage, Docker deployment, multi-node scaling, and PDF export, must be treated as planned or unverified unless confirmed in the final code. The report documents the requirements, literature, diagrams, methodology, implementation, testing approach, results interpretation, and future enhancements.

Keywords: API Load Testing, Chaos Injection, Delay Injection, Performance Monitoring, Real-Time Telemetry, Worker Threads, Node.js, React, Socket.IO, Latency, Throughput, Error Rate

Mapping with Sustainable Development Goals (Mention the Goal)	SDG 4 Quality Education SDG 9 Industry, Innovation and Infrastructure

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ACKNOWLEDGMENT	**VIII**
	ABSTRACT	**IX**
	TABLE OF CONTENTS	**X**
	LIST OF FIGURES	**XII**
	LIST OF ABBREVIATIONS	**XIII**
CHAPTER 1: INTRODUCTION		1
	1.1 Background	1
	1.2 Motivation	2
	1.3 Problem Statement	2
	1.4 Objectives	3
	1.5 Scope and Boundaries	3
	1.6 Organisation of the Report	4
CHAPTER 2: REQUIREMENTS AND ANALYSIS		5
	2.1 Existing System and Limitations	5
	2.2 Proposed System	5
	2.3 Functional Requirements	6
	2.4 Non-Functional Requirements	6
	2.5 Hardware and Software Requirements	7
	2.6 Feasibility Study	8
	2.7 Risk Analysis and Mitigation	8
CHAPTER 3: LITERATURE REVIEW		9
	3.1 Introduction to the Review	9
	3.2 Paper 1 — Chaos Engineering	9
	3.3 Paper 2 — The Tail at Scale	10
	3.4 Paper 3 — Site Reliability Engineering	10
	3.5 Paper 4 — Release It!	10
	3.6 Paper 5 — Worker Threads in Node.js	11
	3.7 Paper 6 — Socket.IO Real-Time Communication	11
	3.8 Paper 7 — Chart.js Visualisation	11
	3.9 Paper 8 — HTTP Semantics	12
	3.10 Paper 9 — Load Testing and Performance Evaluation	12
	3.11 Paper 10 — Software Reliability Measurement	12
	3.12 Comparative Summary and Research Gap	13
CHAPTER 4: DIAGRAMS		14
	4.1 Data Flow Diagram	14
	4.1.1 DFD Level 0	14
	4.1.2 DFD Level 1	15
	4.1.3 DFD Level 2 – Load Generation and Chaos Injection	16
	4.2 Class Diagram	17
	4.3 State Diagram	18
	4.4 Sequence Diagram	19
	4.5 ER Diagram	20

CHAPTER 5: METHODOLOGY 21
5.1 Development Methodology 21
5.2 System Architecture 21
5.3 Module 1 — User Interface and Configuration 22
5.4 Module 2 — Backend Control API 24
5.5 Module 3 — Load Generation Worker 25
5.6 Module 4 — Chaos Delay Injection 25
5.7 Module 5 — Metrics and Real-Time Telemetry 26
5.8 Module 6 — Charts and Summary 28
5.9 Algorithms 29
5.10 Case Study Design 32
CHAPTER 6: IMPLEMENTATION DETAILS 33
6.1 Development Environment and Setup 33
6.2 Frontend Implementation Procedure 33
6.3 Backend Implementation Procedure 33
6.4 Worker Implementation Procedure 34
6.5 Metric Definitions and Formulas 34
6.6 Testing Strategy 35
6.7 Test Cases 35
6.8 Testing Results: How to Report Them 35
6.9 Implementation Limitations 36
CHAPTER 7: ADVANTAGES AND DISADVANTAGES 37
7.1 Advantages 37
7.2 Disadvantages 37
7.3 Comparison with Established Tools 38
7.4 Ethical and Safe Use 38
CHAPTER 8: RESULTS AND DISCUSSION 38
8.1 Results Overview 38
8.2 Interpreting the Metrics 39
8.3 Demonstration Screenshot Values 39
8.4 Proposed Experiment Matrix 40
8.5 Discussion 40
CHAPTER 9: CONCLUSION AND FUTURE SCOPE 41
9.1 Conclusion 41
9.2 Limitations 41
9.3 Future Enhancements 42
9.4 Applications 42
CHAPTER 10: APPENDICES 43
10.1 Sample Code 43
REFERENCES 44

LIST OF FIGURES

Figure No.	Title	Page
Fig. 4.1	DFD Level 0	14
Fig. 4.2	DFD Level 1	15
Fig. 4.3	DFD Level 2 (Process 2.0)	16
Fig. 4.4	Class Diagram	17
Fig. 4.5	State Diagram of a Test Session	18
Fig. 4.6	Sequence Diagram	19
Fig. 4.7	ER Diagram	20
Fig. 4.8	System Architecture	22
Fig. 4.9	User Interface	24
Fig. 5.0	Backend API	25
Fig. 5.1	Metrics	27
Fig. 5.2	Summary	28
Fig. 5.3	Flow Chart	31
Fig. A.1	Frontend Sample Code	43
Fig. A.2	Backend Sample Code	44

LIST OF ABBREVIATIONS

Abbreviation	Meaning
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
UI	User Interface
UX	User Experience
VU	Virtual User
RPS	Requests per Second
CPU	Central Processing Unit
RAM	Random Access Memory
DFD	Data Flow Diagram
UML	Unified Modeling Language
SUT	System Under Test
SLO	Service-Level Objective
SLA	Service-Level Agreement
WS	WebSocket
JSON	JavaScript Object Notation
REST	Representational State Transfer
CI/CD	Continuous Integration / Continuous Delivery
PO	Program Outcome
PSO	Program-Specific Outcome
SDG	Sustainable Development Goal

CHAPTER 1: INTRODUCTION

1.1 Background

Modern applications expose many of their capabilities through Application Programming Interfaces (APIs). An API allows one software component to request a service from another component using a defined interface. In web systems, these requests commonly use HTTP or HTTPS and return structured responses such as JSON. APIs support login, product search, payment, inventory, booking, reporting, and communication between microservices. As an application grows, the number and frequency of API requests can rise quickly, making performance and reliability important parts of software quality.

A service that works correctly for one user may behave differently when hundreds of requests arrive at the same time. Requests may wait in queues, compete for CPU time, consume connection pools, or wait for database and network resources. Average response time alone may hide slow outliers; error rate and throughput are also needed to understand system behaviour. Performance testing provides a controlled way to observe these characteristics before a system is exposed to unpredictable demand.

SimuNet is a browser-based API load-testing and chaos-injection dashboard. It provides a visual interface for configuring a target URL, a virtual-user/request load value, and an artificial delay. The backend uses Node.js and Express to receive test commands and uses a worker thread to issue HTTP requests through Axios. Socket.IO communicates live metric updates to the browser, where chart components can display the evolving measurements.

The project is intended as an educational mini-project that demonstrates the relationship between a user interface, a backend service, concurrent request generation, real-time event communication, and performance metrics. It is not presented as a replacement for a mature enterprise test platform. Rather, it provides a focused environment in which students can observe the effects of concurrency and added delay on an API endpoint.

1.2 Motivation

The motivation for SimuNet comes from the gap between understanding performance testing in theory and observing it in practice. A learner can read about throughput, latency, errors, and resilience, but the concepts become easier to understand when the measurements change while a test is running. A dashboard makes the test configuration visible and reduces the need to inspect raw terminal output.

Traditional performance tools are useful and feature-rich, but their configuration and reporting models may require time to learn. A small browser interface can be a convenient teaching aid because it exposes a limited set of controls and visualises selected metrics. The project therefore focuses on accessibility, immediate feedback, and a clear connection between configuration choices and observed responses.

The chaos-engineering aspect adds another motivation. Systems are often tested under normal conditions but fail when delays, timeouts, or partial failures occur. Controlled delay injection is a simple starting point for studying resilience. It can show how additional waiting time affects request completion and can help explain why reliability must be assessed under more than one operating condition.

The project also offers practical experience with asynchronous JavaScript, worker-thread lifecycle management, HTTP clients, REST-style control endpoints, Socket.IO events, and chart rendering. These topics are relevant to web development and systems engineering because they combine programming, architecture, measurement, and responsible experimentation.

1.3 Problem Statement

Developers need a convenient way to generate concurrent HTTP requests against a test API and observe request outcomes while the test is active. A purely manual approach cannot reliably reproduce a large number of requests, and static output may not reveal how metrics change over time. A test harness must also make it possible to stop a run and summarise the collected measurements.The problem addressed by SimuNet is the design and implementation of a compact web-based dashboard that accepts a target endpoint and load parameters, starts a controlled request-generation process, streams per-interval measurements to a browser, and returns a final summary when stopped. A related learning objective is to introduce configurable artificial delay so

that users can compare behaviour with and without an injected wait.

The solution must remain bounded and understandable. It should be used against local services, staging systems, or other endpoints for which the operator has explicit permission. The current code is a prototype and does not include every safeguard expected in a production load-testing platform; therefore, the report records those limitations and recommends stronger validation, rate caps, authentication, and safety controls for future work.

1.4 Objectives

The primary objective is to develop SimuNet as a web-based interface for starting, observing, and stopping an API load test. The dashboard is intended to make test parameters visible and to display metrics such as successful requests, errors, latency, and approximate request rate. The backend separates the browser interface from request generation so that test work is not performed directly by the browser.

The technical objectives are to implement a React interface, create backend control routes using Express, execute request-generation logic in a Node.js worker thread, send HTTP requests with Axios, and emit measurement messages through Socket.IO. The project also aims to support dynamic updates to virtual-user and delay values while a run is active, subject to the implementation's current limitations.

The analytical objectives are to explain how load, latency, and errors should be interpreted; define a repeatable test procedure; identify the difference between a demonstration screenshot and a controlled benchmark; and document areas requiring additional validation. The academic objectives are to apply software engineering methods, prepare system diagrams, review relevant literature, and document implementation and test cases clearly.

1.5 Scope and Boundaries

The current scope includes a browser-based configuration interface, a backend service listening on a local port, a test-start endpoint, a test-stop endpoint, an update-parameters endpoint, a worker thread that sends HTTP GET requests, an optional artificial delay before a request, and live metrics delivered through Socket.IO. The source also includes a local ping endpoint that can act as a controlled target for a basic demonstration.

The current scope does not establish a production-grade distributed testing service. Multi-node

coordination, durable historical storage, user accounts, role-based access, robust session isolation, a completed PDF export workflow, and a full security model are not confirmed by the supplied backend package and server code. The presentation identifies multi-node scaling and PDF report export as pending work. Redis, Docker, and Tailwind CSS are mentioned in the presentation but should only be described as implemented if the final code and runtime setup demonstrate their use.
The system is best suited for learning, local demonstrations, and controlled testing of endpoints that the operator owns or is authorised to test. It should not be used to send high-volume traffic to public services without permission. The results are affected by the test machine, network, target server, request concurrency, and implementation details, so a single run cannot establish universal performance capacity.

1.6 Organisation of the Report

Chapter 1 introduces the topic, motivation, problem, objectives, scope, and report organisation. Chapter 2 analyses the existing approach, proposed solution, functional and non-functional requirements, feasibility, assumptions, and risks. Chapter 3 reviews literature and technical documentation related to chaos engineering, load testing, concurrent execution, telemetry, and visualisation.

Chapter 4 presents the system diagrams, including context and data-flow views, use cases, component structure, activity flow, sequence flow, and session-state transitions. Chapter 5 explains the methodology, architecture, modules, algorithms, and a controlled case-study design. Chapter 6 documents implementation procedures, setup, operational workflow, and a testing plan. Chapter 7 discusses advantages and disadvantages. Chapter 8 presents analysis and interpretation, Chapter 9 concludes the work and outlines limitations and future scope, and Chapter 10 contains appendices and selected code excerpts.

CHAPTER 2: REQUIREMENTS AND ANALYSIS

2.1 Existing System and Limitations

Performance testing can be carried out using command-line scripts, desktop load-testing applications, API clients, or custom automation. These approaches are valuable, but a learner may need to create scripts, configure test plans, parse logs, or combine multiple tools to see a live picture of request behaviour. A simple dashboard can bring the key controls and selected measurements into one view.

Manual API testing is appropriate for checking correctness of individual requests, but it does not by itself represent concurrent traffic. Repeating requests manually is slow and inconsistent. A command-line loop can generate more traffic, yet it may provide limited visual feedback unless logging and aggregation are added. A browser-based tool can help connect configuration to observed metrics while still relying on a backend for actual request generation.

The limitations described here are general observations, not claims that existing tools are incapable of real-time charts or distributed testing. Established tools such as Apache JMeter and other commercial or open-source products have broader capabilities than this mini-project. SimuNet's contribution is its small educational workflow and its focus on a few understandable parameters rather than feature parity with mature tools.

2.2 Proposed System

SimuNet provides a frontend where a user enters a target URL and chooses load and artificial delay parameters. The frontend sends a command to the Express backend. The backend creates a worker thread using the provided test parameters and responds that the test has been initiated. The worker periodically generates a batch of asynchronous HTTP GET requests and sends a metrics message to the parent thread when the batch completes.

The backend aggregates incoming worker messages into session-level counters and broadcasts the interval metrics through Socket.IO. The browser can use these events to update live visualisations. When the user stops the test, the backend terminates the active worker and calculates a summary containing duration, total requests, peak requests in an interval, average latency for successful requests, and error percentage.

The implementation uses a compact in-memory model. It does not persist all sessions to a database. A production version would require stronger request validation, explicit limits on concurrency and duration, worker error handling, isolated session state, a permission model, and a reliable approach to metric aggregation. The report therefore distinguishes the implemented prototype from the proposed improvements.

2.3 Functional Requirements

FR1: The user should be able to enter a target URL. The backend should validate that the value is present and is a supported HTTP or HTTPS URL before starting a test. The current source accepts parameters from the request body with limited validation, so robust validation is a recommended improvement rather than a confirmed implemented behaviour.

FR2: The interface should allow a load parameter representing virtual users or concurrent requests per worker batch to be selected. FR3: The interface should allow an artificial delay value to be selected. FR4: The frontend should request that the backend start a test. FR5: The backend should create the request-generation worker and return a start response. FR6: The worker should issue HTTP requests and count successful completions and errors.

FR7: The worker should report interval metrics to the main thread. FR8: The server should publish the metrics to connected dashboard clients through Socket.IO. FR9: The user should be able to request a stop operation. FR10: The backend should terminate active worker threads and return a final summary. FR11: The system should support dynamic update messages for load and latency values while the worker is running. These requirements describe the intended workflow; the actual browser UI and final integration should be checked against the submitted build.

2.4 Non-Functional Requirements

Performance: the dashboard should remain responsive while metrics are received, and the backend should avoid doing all request-generation work in the main event loop. The use of a worker thread creates a separate JavaScript execution thread, but it does not automatically guarantee unlimited throughput. Network I/O, target capacity, CPU, sockets, and the design of the request loop remain important constraints.

Reliability: a test should stop predictably, worker errors should be detected, and final counters

should be internally consistent. Usability: the dashboard should label each control and metric clearly and should show whether a run is active. Maintainability: modules and API routes should have understandable responsibilities. Compatibility: the frontend and backend must use compatible runtime versions and configured addresses. Security: the application should be restricted to authorised test targets and should avoid exposing an unrestricted traffic generator to untrusted users.

Observability: the system should make metrics available during the run and provide a final summary. Measurement accuracy: the definitions of total requests, successful requests, errors, latency, duration, and peak rate must be explicit. The current implementation records latency only for successful requests and uses interval counts to estimate peak request rate; the report should not describe these metrics as percentiles or as a fully validated requests-per-second measurement without further testing.

2.5 Hardware and Software Requirements

The software can be developed on a computer capable of running a supported Node.js version, npm, a modern browser, and a code editor. The exact minimum CPU, RAM, operating system, and browser versions depend on the test load and must be filled in from the actual development environment. Do not invent hardware specifications: record them using system information tools before final submission.

The source packages identify React 19.2.8, Vite 8.2.0, Chart.js 4.5.1, react-chartjs-2 5.3.1, lucide-react, and socket.io-client 4.8.3 on the frontend. The backend package identifies Express 5.2.1, Socket.IO 4.8.3, Axios 1.19.x, CORS 2.8.6, and nodemon 3.1.14. Package ranges are not the same as the exact installed versions; use the lock files and npm commands to record precise versions for the final report.

The backend package does not list Redis or Docker as dependencies, and the provided server file does not connect to Redis. The frontend package does not list Tailwind CSS. These tools may appear in planning material or slides, but should not be listed as deployed dependencies unless the final repository includes and uses them. The application starts the backend on port 4000, while the Vite development server supplies the frontend during development.

2.6 Feasibility Study

Technical feasibility is supported by the availability of mature JavaScript libraries for the browser, HTTP server, HTTP client, worker threads, and real-time event communication. The main technical challenges are bounded concurrency, correct metric aggregation, worker lifecycle management, safe URL handling, and keeping the browser and backend in sync. A local test endpoint reduces external dependencies during initial validation.

Economic feasibility is favourable for a student prototype because the core packages are open-source and the test can run locally. Costs may arise if cloud infrastructure, hosted targets, monitoring, or additional development environments are used. Operational feasibility depends on a straightforward interface, clear metric labels, and simple start/stop actions. Legal and ethical feasibility requires that tests be limited to owned or explicitly authorised endpoints.

Schedule feasibility can be evaluated through the six-sprint plan in the project presentation: requirements, architecture, load engine, telemetry, chaos injection, and benchmarking. The presentation reports several core components as completed and lists PDF export and multi-node scaling as pending. The final report should use the actual completion status at submission time and should not state that a pending feature is complete.

2.7 Risk Analysis and Mitigation

An unrestricted target field creates a risk that users may accidentally direct load to a third-party system. The safest approach is to default to a local test endpoint, validate URLs, restrict private/public target policy as appropriate, require confirmation for non-local targets, and document that permission is mandatory. A production deployment should not expose the test-start endpoint without authentication and abuse controls.

Excessive concurrency can consume memory, sockets, CPU, and bandwidth or overload the target. Mitigations include upper bounds on virtual users, duration caps, cancellation, per-target rate limits, request timeouts, and back-pressure. A worker that crashes or never returns can leave the interface in an inaccurate state, so the backend should handle worker error and exit events and transition the session to a stopped or failed state. Metric errors can mislead decisions.

The implementation should define whether request counts are based on attempts or completions, count HTTP non-success status codes separately if needed, use high-resolution timing where appropriate, and validate formulas with known cases. Socket disconnections and browser refreshes should not cause a test to run indefinitely. Finally, test results should record environment details and repeat runs because network and server variability can change measurements.

CHAPTER 3: LITERATURE REVIEW

3.1 Introduction to the Review

This chapter reviews established work and authoritative documentation relevant to SimuNet. The topics include chaos engineering, tail latency, reliability engineering, load-testing methodology, concurrency, event-based communication, chart-based visualisation, and HTTP semantics. The review is used to justify design choices and identify limitations; it does not imply that SimuNet implements all features described by each source.

A critical review should distinguish peer-reviewed research from books, standards, and technical documentation. Academic papers provide concepts and empirical findings, while official documentation explains the behaviour and correct use of the technologies selected for implementation. The ten entries below are a combination of research papers and foundational technical sources. The institution's instruction says to add ten papers; if the guide requires ten peer-reviewed papers only, replace documentation entries with papers approved by the guide before submission.

3.2 Paper 1 — Chaos Engineering

Basiri and colleagues' article “Chaos Engineering” discusses the need to study complex software systems by deliberately introducing realistic failure conditions and observing system-level behaviour. The central idea is that reliability cannot be inferred only from a system's design or from normal-path tests. Controlled experiments can expose hidden assumptions about timeouts, dependencies, and recovery behaviour.

Relevance to SimuNet: the project introduces a configurable delay before sending a request. This is a limited form of fault-condition simulation and can help demonstrate how latency affects a test run. It should not be described as a complete chaos-engineering framework because the current code does not implement a broad catalogue of failures, automated hypotheses, rollback policies, or production safeguard.Limitation and lesson: chaos experiments should have a clear hypothesis, a bounded blast radius, and observable outcomes. For SimuNet, a suitable hypothesis is that increasing the injected delay will increase request completion time and may reduce the number of completed requests in a fixed observation window. This hypothesis should be tested on a local controlled endpoint and recorded

3.3 Paper 2 — The Tail at Scale

Dean and Barroso's "The Tail at Scale" examines the effect of high-percentile latency in large-scale interactive services. Even when most requests complete quickly, a small number of slow operations can affect user-visible response time when services compose multiple dependencies. The work highlights why averages alone may be insufficient to describe a system's performance.
Relevance to SimuNet: the dashboard currently reports average latency for successful requests in the worker interval and a final average. This is useful for a basic demonstration, but it cannot describe tail behaviour. A future version should retain individual latency samples or a histogram and calculate p50, p90, p95, and p99 percentiles. Percentile reporting must be implemented and validated before it is claimed as a feature.

3.4 Paper 3 — Site Reliability Engineering

Beyer and colleagues' Site Reliability Engineering book explains practical approaches to reliability, service-level indicators, service-level objectives, monitoring, incident response, and capacity planning. The text promotes measurable reliability rather than relying only on subjective impressions. Metrics are meaningful when their definitions and operational context are clear.

Relevance to SimuNet: requests per second, error percentage, and latency can act as simple indicators for a controlled test. However, a mini-project dashboard does not itself define service-level objectives or guarantee that a target meets a reliability target. The report should explain the metrics and avoid turning one short run into a general claim about production readiness.

3.5 Paper 4 — Release It!

Michael Nygard's book Release It! discusses stability patterns and failure modes in production systems, including timeouts, circuit breakers, bulkheads, and resource exhaustion. Its practical focus is relevant because a load generator and its target can both fail when concurrency or waiting time is poorly controlled.

Relevance to SimuNet: the current worker creates a batch of requests and waits for the batch to complete before posting metrics. Request timeouts and bounded batch sizes are important safeguards. A more mature implementation could add abortable requests, maximum duration, worker shutdown acknowledgement, and clear failed-session states. These are recommendations unless they are confirmed in the final code.

3.6 Paper 5 — Worker Threads in Node.js

The official Node.js worker_threads documentation explains that workers create separate JavaScript execution threads and can be useful for CPU-intensive JavaScript work. It also notes that built-in asynchronous I/O is generally efficient for I/O-intensive work. This distinction matters when describing the architecture accurately.

Relevance to SimuNet: the backend creates a worker thread that runs the request loop. The worker executes asynchronous Axios requests, so the project combines a worker thread with asynchronous network I/O. It is accurate to say that the worker is separate from the main thread; it is not accurate to claim that worker threads alone make HTTP I/O faster or guarantee a particular request rate.

3.7 Paper 6 — Socket.IO Real-Time Communication

Socket.IO documentation describes a low-latency, bidirectional, event-based communication layer between a client and a server. It supports named events and connection handling and may use WebSocket or other transport mechanisms depending on the environment. The event model is convenient for pushing updates without requiring the browser to repeatedly poll for each metric.
Relevance to SimuNet: the server emits a `live-metrics` event for each worker metrics message. The dashboard can listen for this event and update its charts. A production-ready system should define a stable event schema, handle disconnect and reconnect cases, avoid sending excessive updates, and isolate metrics by test session if multiple users or sessions are supported.

3.8 Paper 7 — Chart.js Visualisation

Chart.js provides common chart types and supports browser-based data visualisation. Line charts are particularly useful for displaying values over time, while bar charts can compare categories. The choice of chart should follow the question being answered rather than being decorative.

Relevance to SimuNet: a time-series graph can show how interval success counts, errors, or latency change during a run. The chart should have labelled axes, a clear unit, and a bounded number of displayed points so that a long session does not create unnecessary rendering overhead. If the current chart uses a rolling window, that behaviour should be described only after confirming the frontend code.

3.9 Paper 8 — HTTP Semantics

The HTTP Semantics specification defines the meaning of HTTP methods, status codes, headers, and request/response behaviour. Load testing depends on consistent interpretation of responses. A request that receives an HTTP error status is different from a request that fails at the network layer, and both should be defined clearly in the metrics model.

Relevance to SimuNet: the worker uses Axios to issue GET requests and counts resolved requests as successful while counting rejected promises as errors. Depending on Axios configuration, HTTP response status codes outside its accepted range may reject the promise, but the report should still describe the actual configured behaviour rather than assume all response classes are separately tracked. A future test engine could report status-code groups and network exceptions separately.

3.10 Paper 9 — Load Testing and Performance Evaluation

Performance evaluation is strongest when tests are repeatable and the workload is described. A test plan should identify the target environment, request type, load level, duration, warm-up period, injected conditions, and metrics. Repeating a run helps reveal whether a result is stable or an outlier. Tests should also avoid changing multiple variables at once when the goal is to understand cause and effect.

Relevance to SimuNet: the virtual-user parameter and delay parameter provide two controllable variables. A basic experiment can hold the target and duration constant while changing one variable. A second experiment can hold the load constant while changing delay. The report should include raw observations and the test environment rather than presenting a graph without a reproducible procedure.

3.11 Paper 10 — Software Reliability Measurement

Software reliability engineering emphasises identifying failure modes, measuring their frequency and impact, and using evidence to guide improvement. A test harness contributes to this process by collecting measurements under defined conditions. The quality of conclusions depends on the quality of the measurement process and on whether the test workload resembles the intended use case.

Relevance to SimuNet: the project provides a useful entry point for observing throughput, latency,

and errors. The present prototype has limitations in metric precision, request validation, and persistence. These limitations should be acknowledged so that the dashboard is used as a learning tool rather than as the sole basis for production capacity decisions.

3.12 Comparative Summary and Research Gap

Across the reviewed sources, three themes recur. First, reliability requires controlled experiments and meaningful metrics. Second, concurrent workloads can reveal bottlenecks that are not visible during manual testing. Third, real-time visualisation makes it easier to understand a changing system, but visualisation cannot compensate for inaccurate measurement or an unsafe test design.

The gap addressed by SimuNet is an educational integration of request generation, configurable delay, live event streaming, and a browser dashboard in a small codebase. Its value lies in the visibility of the testing workflow. Its limits are equally important: the project does not yet provide distributed load generation, durable historical records, comprehensive fault types, production-grade safeguards, or percentile analytics. These gaps define practical future work.

CHAPTER 4: DIAGRAMS

4.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) shows how data enters the system, how it is processed, where it is stored and how the results are returned to the users. In SimuNet the two external entities are the Developer / Tester, who configures and controls the test, and the Target API / Server, which receives the generated load. The DFD is presented at three levels: Level 0 gives the system context, Level 1 splits the system into its main processes, and Level 2 expands the load generation and chaos injection process.

4.1.1 DFD LEVEL

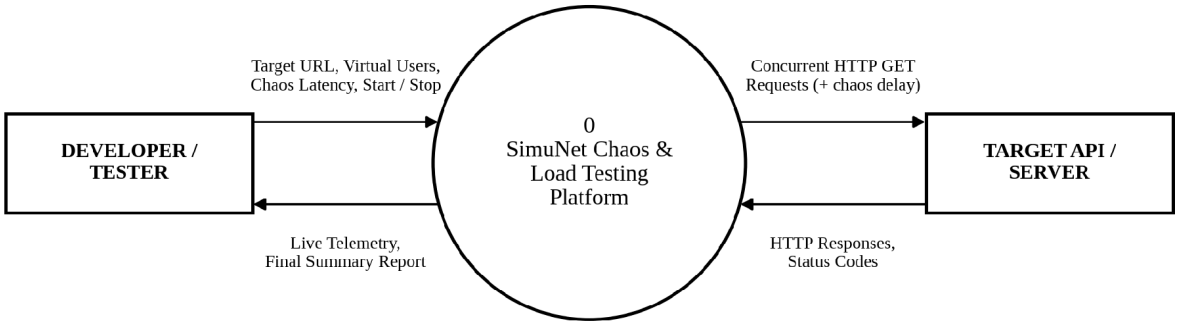
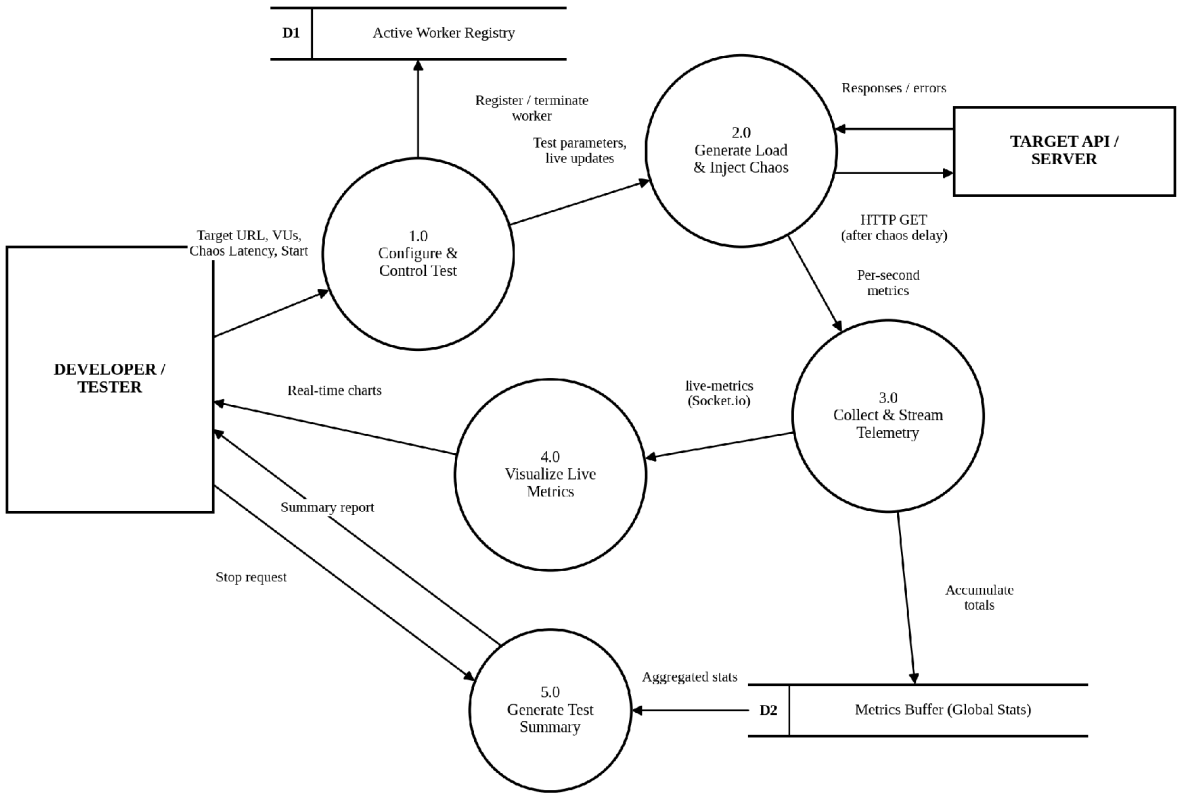


Fig. 4.1 DFD Level 0

The Level 0 diagram (context diagram) represents SimuNet as a single process. The Developer / Tester supplies the target URL, the number of virtual users (VUs), the chaos latency and the start or stop command. SimuNet returns live telemetry while the test runs and a final summary report when it ends. Towards the Target API, SimuNet sends concurrent HTTP GET requests, optionally delayed by the chaos latency, and receives the HTTP responses and status codes, which are used to measure latency and errors.



4.1.2 DFD LEVEL 1

Fig. 4.2 DFD Level 1

1. Level 1 divides SimuNet into five processes. Process 1.0 (Configure and Control Test) accepts the target URL, VUs and chaos latency, registers or terminates the worker in the Active Worker Registry (D1) and forwards the parameters, including live slider updates, to Process 2.0. Process 2.0 (Generate Load and Inject Chaos) runs inside a Node.js worker thread, delays each request by the chaos latency, sends the HTTP requests to the Target API and collects the responses and errors. Process 3.0 (Collect and Stream Telemetry) receives the per-second metrics from the worker, accumulates the totals in the Metrics Buffer (D2) and pushes each sample to the dashboard over Socket.io. Process 4.0 (Visualize Live Metrics) draws the real-time Chart.js graphs for the tester. When the tester stops the test, Process 5.0 (Generate Test Summary) reads the aggregated statistics from D2 and returns the total requests, peak requests per second, average latency and error rate.

4.1.3 DFD LEVEL 2 - LOAD GENERATION AND CHAOS INJECTION

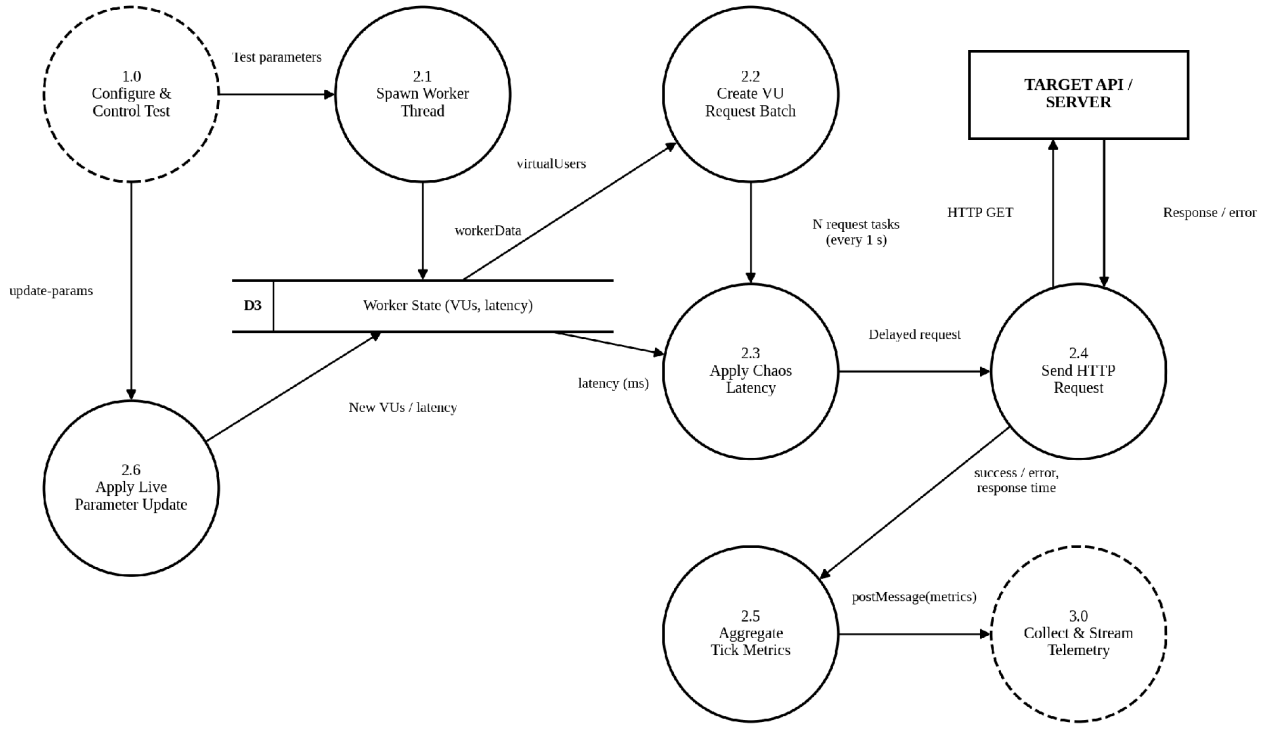


Fig. 4.3 DFD Level 2 (Process 2.0)

Level 2 expands Process 2.0. Sub-process 2.1 spawns the worker thread with the initial workerData. Sub-process 2.2 creates one request task for every virtual user at each one-second tick, using the virtualUsers value held in Worker State (D3). Sub-process 2.3 applies the chaos latency by waiting for the configured number of milliseconds before each request. Sub-process 2.4 sends the HTTP GET request to the Target API and receives the response or error. Sub-process 2.5 counts successes and errors, calculates the average latency and posts the result to Process 3.0. Sub-process 2.6 receives live parameter updates from Process 1.0 and writes the new virtual user and latency values into D3, so that the next tick uses them without restarting the test.

4.2 CLASS DIAGRAM

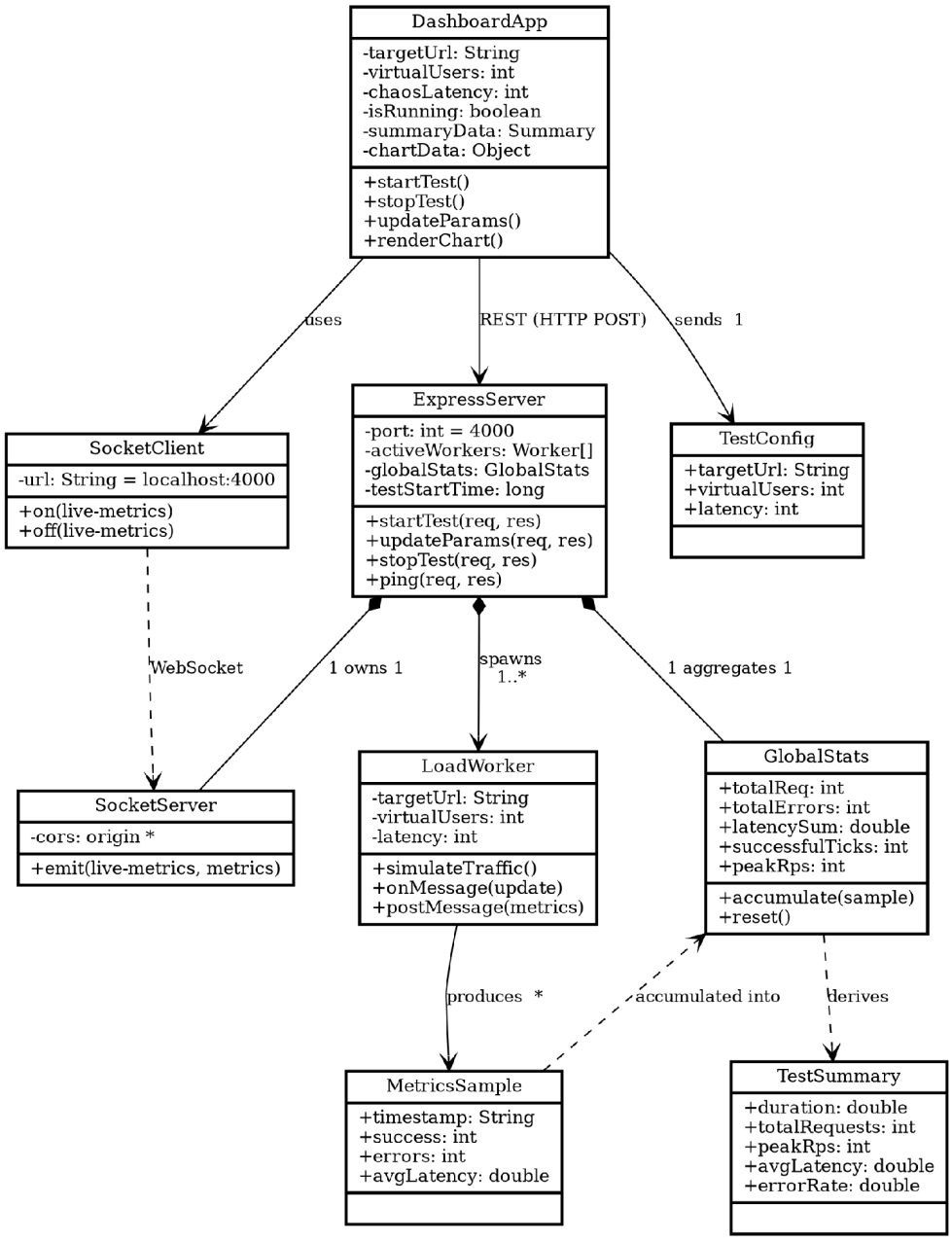


Fig. 4.4 Class Diagram

The class diagram shows the main classes of SimuNet. On the client side, DashboardApp holds the test configuration state (target URL, virtual users, chaos latency), controls the test through REST calls and uses SocketClient to receive live metrics. On the server side, ExpressServer exposes the start-test, update-params, stop-test and ping endpoints, owns the SocketServer and the GlobalStats object, and spawns one or more LoadWorker threads. Each LoadWorker produces MetricsSample objects every second. GlobalStats accumulates these samples, and the TestSummary shown to the user is derived from it when the test is stopped.

4.3 STATE DIAGRAM

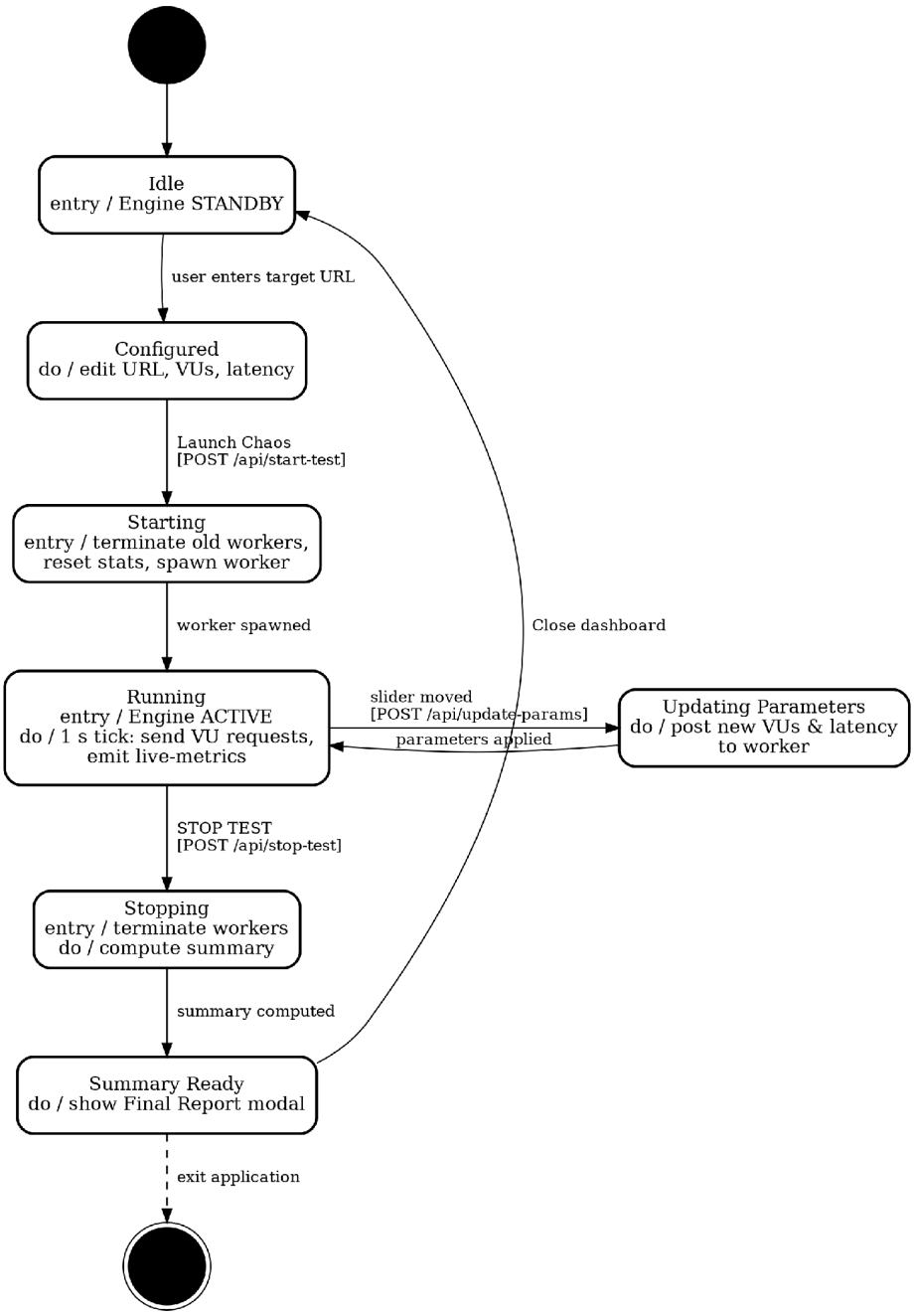


Fig. 4.5 State Diagram of a Test Session

The state diagram describes the life cycle of a test session. The system starts in the Idle state with the load engine on standby. After the tester enters the configuration it moves to Configured. Launch Chaos moves it through Starting, where old workers are terminated, statistics are reset and a new worker is spawned, into Running, where the engine is active and metrics are emitted every second. While running, moving a slider takes the session to Updating Parameters and back to Running without interruption. STOP TEST moves the session to Stopping, where the workers are terminated and the summary is computed, then to Summary Ready, where the Final Report is displayed. Closing the report returns the system to Idle.

4.4 SEQUENCE DIAGRAM

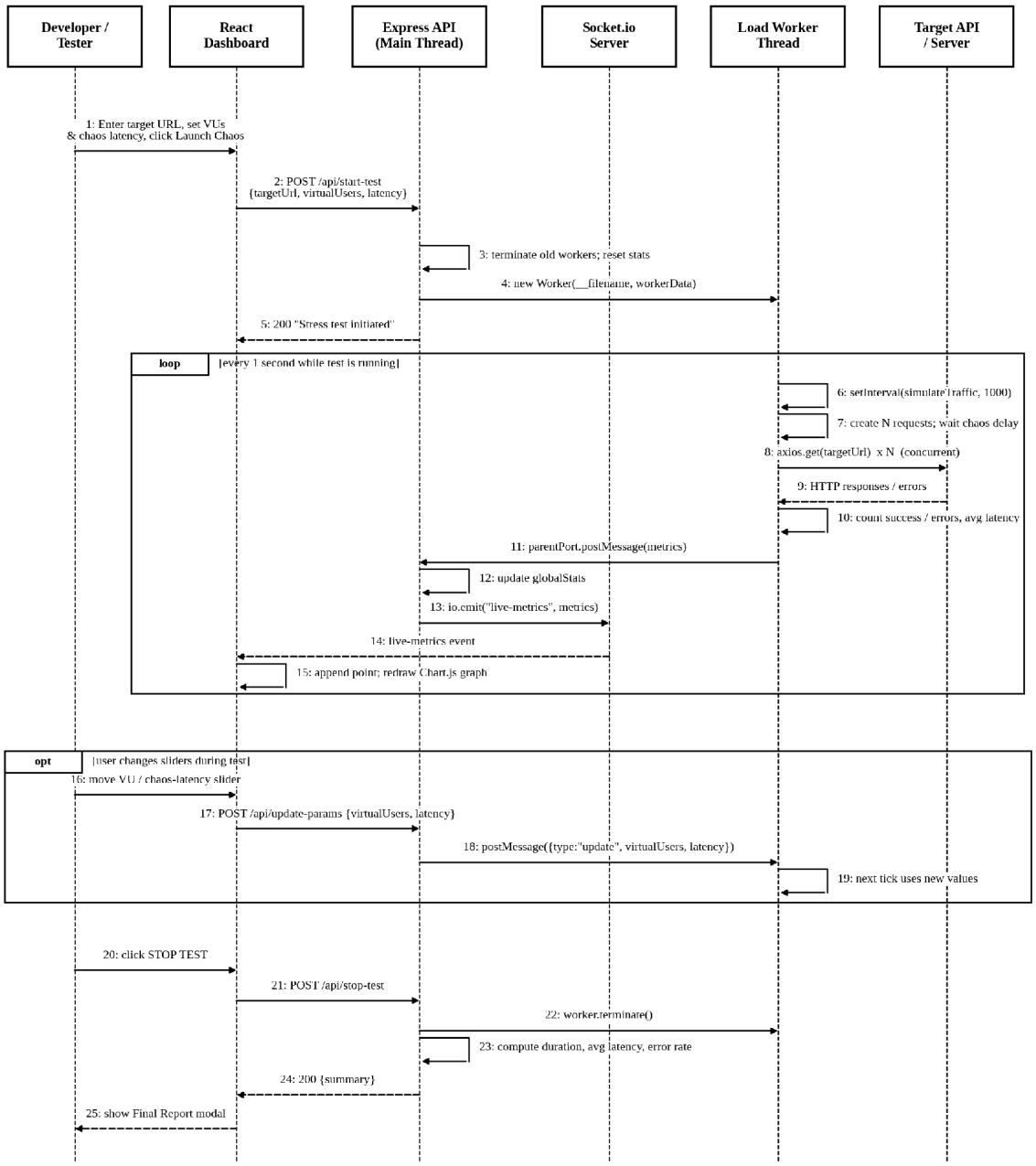


Fig. 4.6 Sequence Diagram

The sequence diagram shows the order of messages between the Developer / Tester, the React Dashboard, the Express API (main thread), the Socket.io server, the Load Worker thread and the Target API. The tester launches the test, the dashboard calls POST /api/start-test and the main thread spawns the worker. In a loop that runs every second, the worker waits for the chaos delay, sends N concurrent requests, counts successes and errors and posts the metrics to the main thread. The main thread updates its global statistics and emits a live-metrics event through Socket.io, and the dashboard redraws the chart. If the tester moves a slider, the new values travel to the worker through POST /api/update-params. On STOP TEST the worker is terminated and the summary is returned to the dashboard.

4.5 ER DIAGRAM

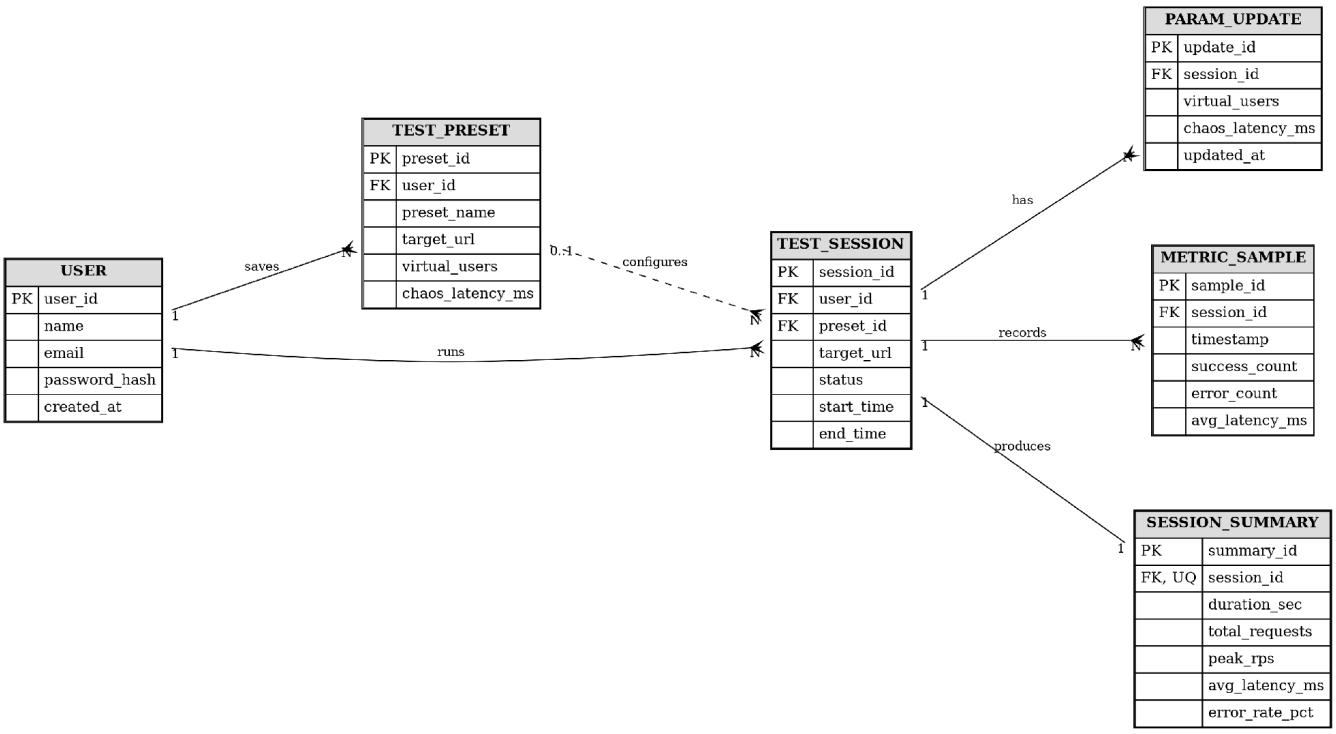


Fig. 4.7 ER Diagram

The ER diagram gives the logical data model of SimuNet. A USER can save many TEST_PRESETs and can run many TEST_SESSIONs. Each TEST_SESSION records many METRIC_SAMPLEs (one per second), may have many PARAM_UPDATEs (live slider changes) and produces exactly one SESSION_SUMMARY when it ends. A TEST_SESSION may optionally be created from a TEST_PRESET. In the current prototype the session statistics are held in memory by the server; this model defines the structure for persistent storage of users, presets and session history.

CHAPTER 5: METHODOLOGY

5.1 Development Methodology

The project presentation describes a six-sprint Agile engineering lifecycle. Sprint 1 focuses on requirements analysis and identifying common difficulties in API load testing. Sprint 2 defines the system architecture and the boundaries between dashboard, backend, worker, and telemetry. Sprint 3 implements the load-generation worker. Sprint 4 implements the telemetry path. Sprint 5 introduces configurable delay injection. Sprint 6 focuses on validation and benchmarking.

The iterative method is appropriate for a compact software project because each sprint produces a visible component that can be reviewed. Requirements can be refined after the first dashboard is available, while integration issues can be discovered before the final stage. The report should include actual sprint dates and work completed from the project log; dates should not be invented solely to make the report look complete.

5.2 System Architecture

The system uses a frontend/backend separation. The frontend collects user input and presents metrics. The backend validates and handles control requests, manages worker lifecycle, aggregates metrics, and broadcasts updates. The worker performs the repeated request workload. The target API is external to the SimuNet process and may be a local service created for testing. This separation allows the user interface to remain responsive while the backend handles the test workflow.

The worker uses a batch-oriented request loop: it creates a collection of asynchronous operations based on the current virtual-user value, optionally waits for the configured artificial latency before each request, awaits the batch, counts successes and errors, calculates average latency for successful requests, and posts a message to the parent thread. The parent aggregates these interval metrics and emits a `live-metrics` event.

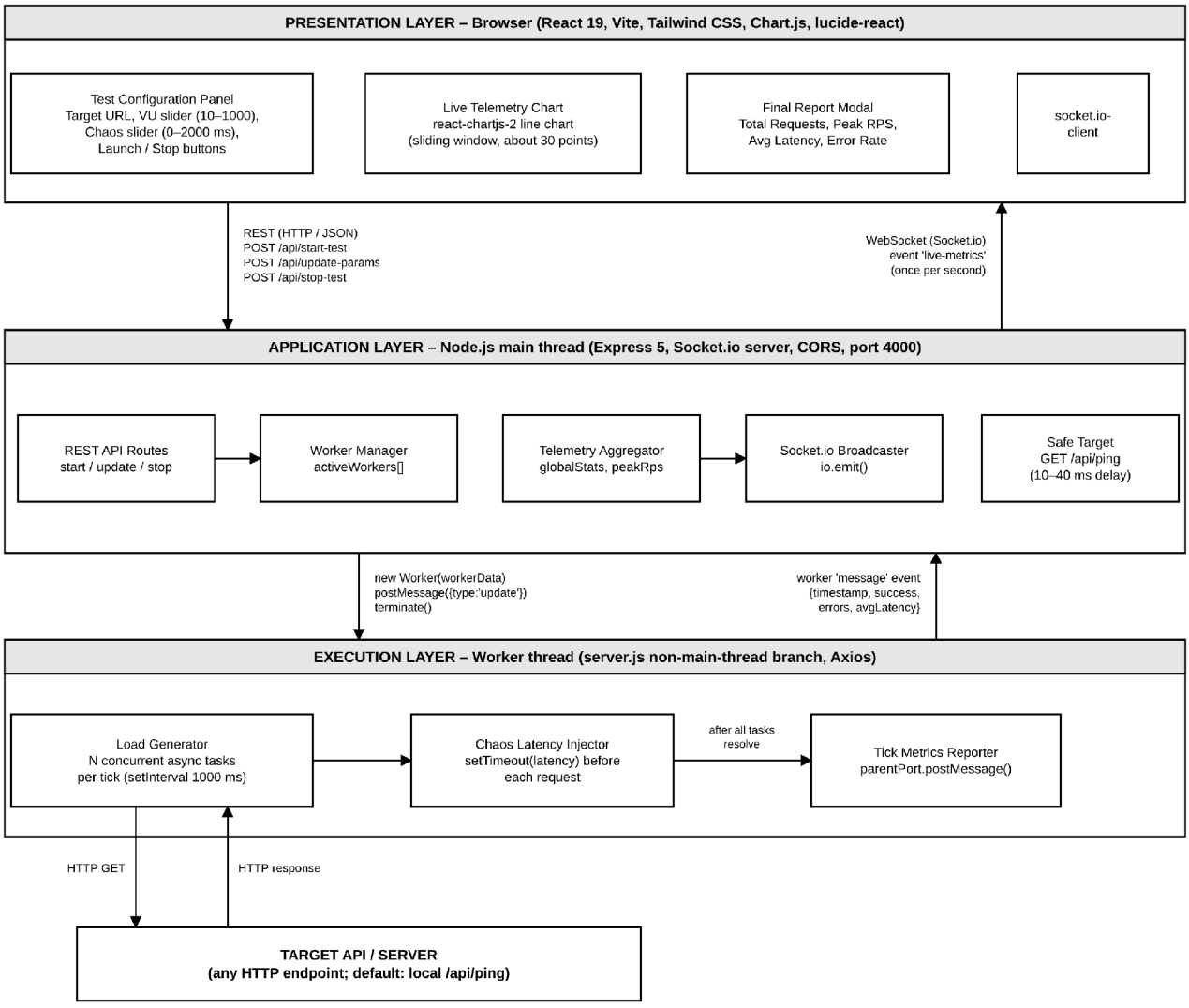


Fig 4.8 architecture

5.3 Module 1 — User Interface and Configuration

The dashboard is the main interaction point. It is intended to display the target URL field, virtual-user control, artificial-delay control, start and stop actions, and performance indicators. A clear interface reduces the possibility of confusing configuration values and helps the operator understand the relationship between inputs and results. Labels should include units, such as milliseconds for artificial delay and requests per second only when the measurement actually represents that unit.

The frontend uses React and Vite. React supports component-based UI composition, while Vite provides the development server and build tooling. The package list also includes Chart.js, react-chartjs-2, lucide-react, and socket.io-client. The exact visual structure and component names should be described from the final frontend source; the project screenshots should be inserted below this module in the final formatted report.

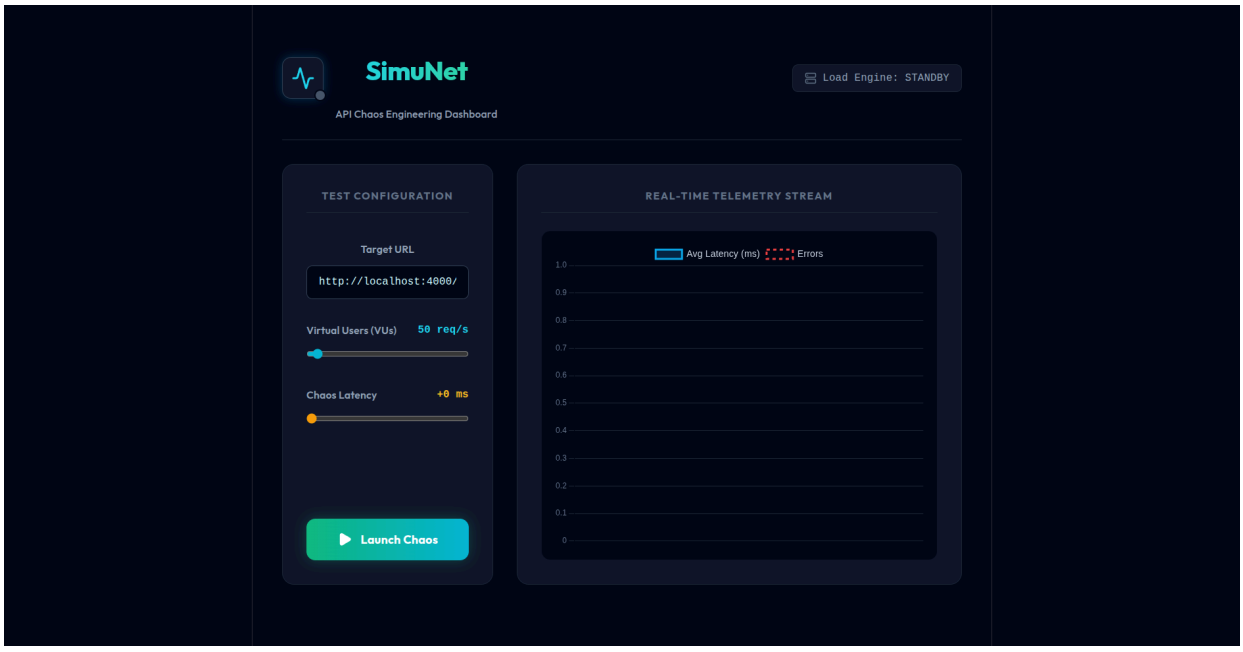


Fig.4.9 user interface

5.4 Module 2 — Backend Control API

The Express backend exposes a local ping endpoint, a test-start endpoint, a parameter-update endpoint, and a test-stop endpoint. The local ping route returns a JSON response after a short random delay, making it useful as a controlled demonstration target. The start route reads target URL, virtual users, and latency from the request body, resets in-memory statistics, creates a worker, and returns a response indicating that the test has been initiated.

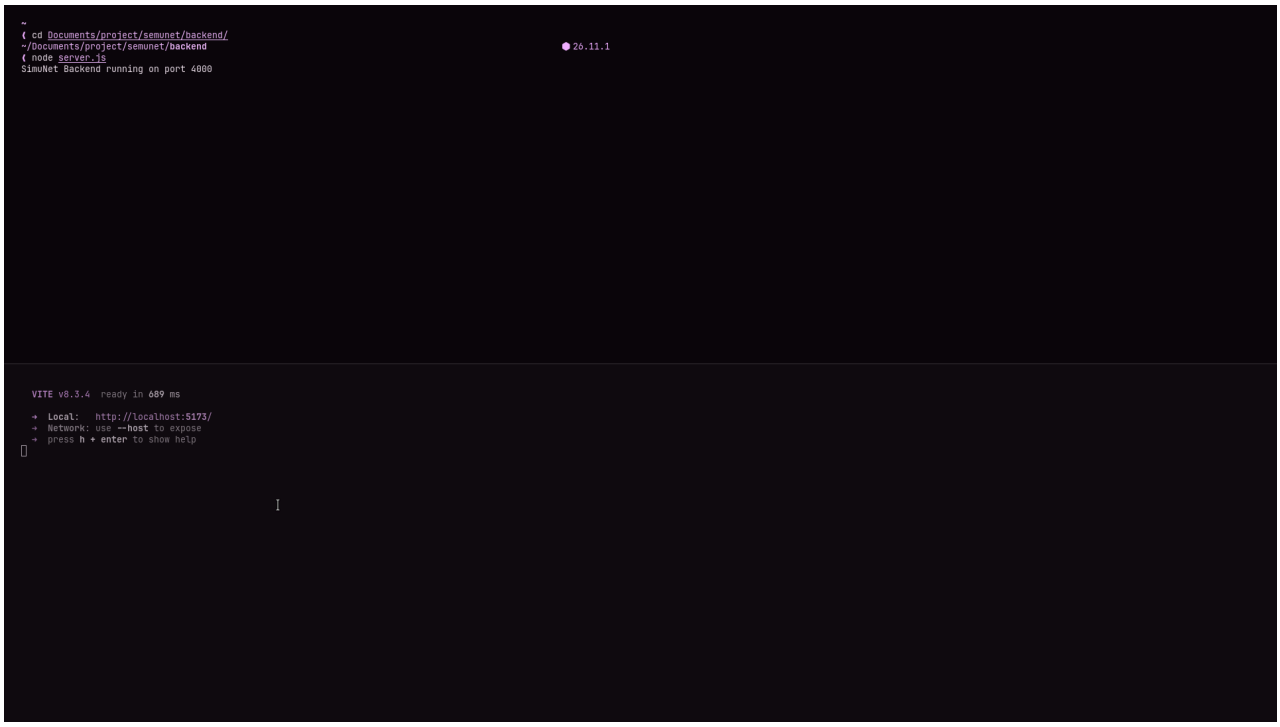


Fig 5.0 backend API

5.5 Module 3 — Load Generation Worker

The worker thread is created using Node.js `worker_threads`. When running as a worker, the server file loads Axios and reads the target URL, virtual-user count, and delay from `workerData`. It listens for update messages and changes the current load and delay values. A periodic loop generates a batch of requests, waits for optional delay, sends GET requests, and posts a metrics object back to the main thread.

The worker's separate execution thread helps separate the request loop from the main server logic, but concurrency is bounded by the configured batch size and the capacity of the target and host. Because the loop waits for all requests in a batch before posting its next metrics message, long-running requests can delay the next interval. The implementation should be reviewed for overlapping interval calls and bounded in-flight requests if high load is expected.

5.6 Module 4 — Chaos Delay Injection

The current chaos mechanism is artificial latency. Before each HTTP request, the worker checks whether the configured latency value is greater than zero. If so, it awaits a timer for that many milliseconds before issuing the request. This makes the request path slower by design and provides a simple way to compare normal and delayed runs.

This mechanism is intentionally narrow. It does not currently demonstrate packet loss, random HTTP failures, dependency outages, DNS failures, or a full resilience experiment framework. In a report, use terms such as “configurable artificial delay” or “basic delay injection” rather than claiming a comprehensive chaos-engineering suite. Future extensions could introduce deterministic failure probabilities, timeout injection, and a clear experiment hypothesis, but only in a controlled environment.

5.7 Module 5 — Metrics and Real-Time Telemetry

The worker reports a timestamp, successful request count, error count, and average latency for successful requests. The main thread calculates total requests for the interval as success plus errors, accumulates error counts and successful latency totals, and tracks the largest interval count as peak RPS. It then emits the metrics object through Socket.IO using the `live-metrics` event.

The final summary includes duration, total requests, peak interval count, average latency, and error rate. The current peak value is based on the number of completed requests reported in a worker

interval; it is an approximation of requests per second because interval timing and request-batch completion affect the count. The current latency accumulator weights the worker's average latency by successful request count, which is appropriate for a weighted mean when each reported average is accurate. More robust metrics would use monotonic timing and explicit per-second buckets.

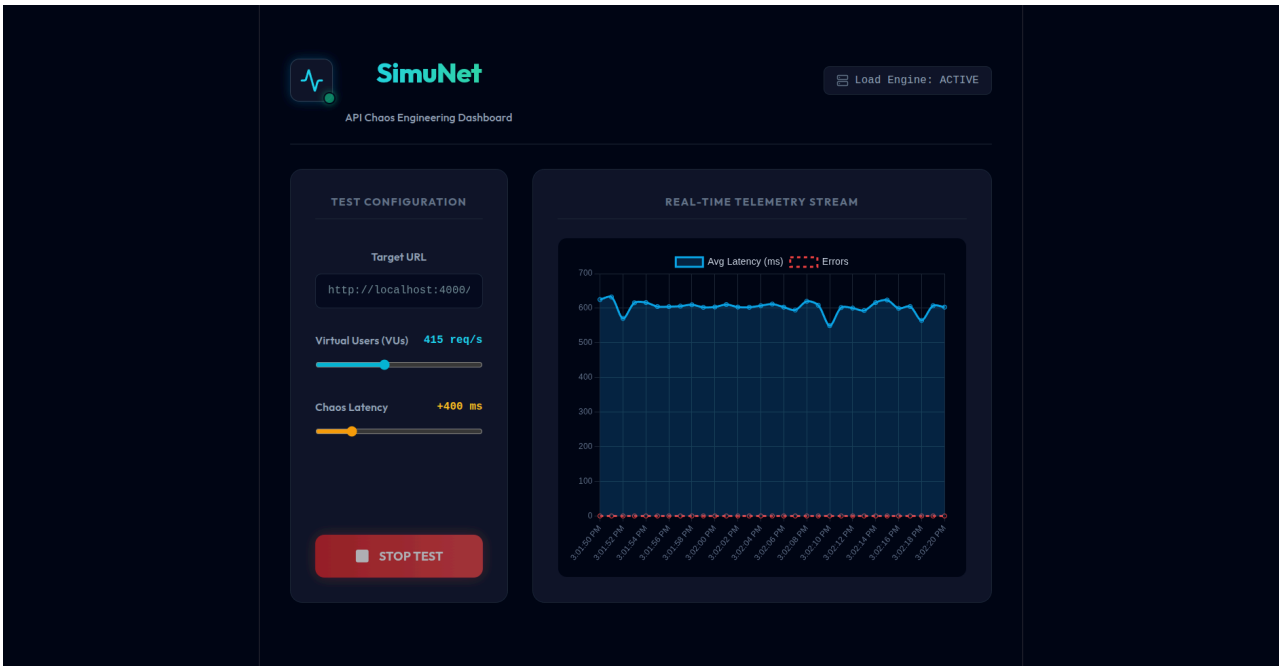


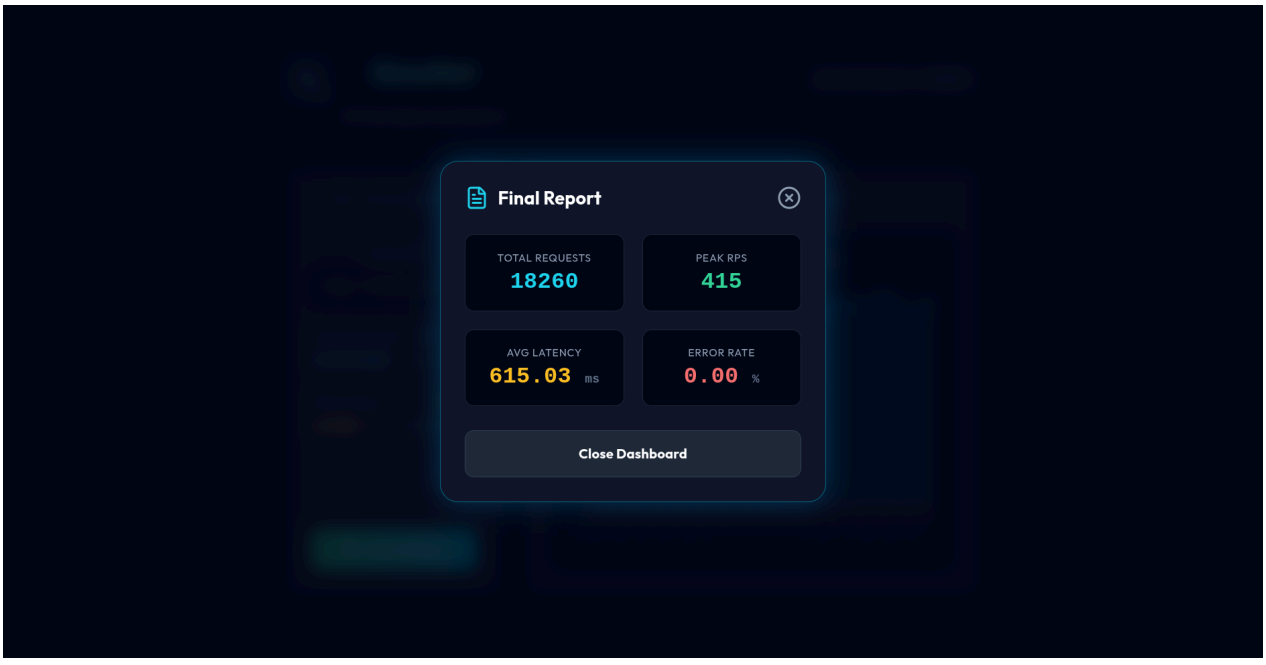
Fig 5.1 Metrics

5.8 Module 6 — Charts and Summary

The frontend package includes Chart.js and react-chartjs-2, which can be used to render changing metrics. A line chart is suitable for time-based values such as latency or completed requests per interval. A metric card can show total requests or error rate. The chart should use meaningful axis labels, consistent units, and readable legends. It should avoid presenting estimates with false precision.

The presentation lists a PDF report exporter as pending. Unless a working export function exists in the final code, describe the final summary as a JSON/API response or on-screen summary rather than a completed downloadable PDF report. A future exporter could include configuration, test environment, run duration, summary metrics, and plots, with an explicit note about whether the results are measured or illustrative.

Fig 5.2 summery



5.9 Algorithms

The behaviour of SimuNet is governed by six algorithms. Five of them are implemented in the Node.js back end (backend/server.js), which contains both the main-thread logic and the worker-thread logic in a single file, and one is implemented in the React front end (frontend/src/App.jsx). The first algorithm initialises a test session, the second generates the virtual-user load and injects chaos latency, the third applies slider changes while a test is running, the fourth aggregates and broadcasts telemetry, the fifth computes the final summary when a test is stopped, and the sixth maintains the live chart. Each algorithm is expressed below as pseudocode that follows the implemented code, and the quantities it uses are defined in Table 5.1.

Parameter	Meaning	Range / Value
targetUrl	URL of the API endpoint that receives the load	Text; default http://localhost:4000/api/ping
virtualUsers (N)	Number of concurrent virtual-user requests created in every tick	10 to 1000 (slider); default 50
latency (L)	Artificial chaos delay added before every request, in milliseconds	0 to 2000 ms, step 50; default 0
Tick interval	Period at which the worker creates a new batch of requests	1000 ms (fixed)
Chart window	Number of earlier points kept on the live chart	30 earlier points plus the newest point
globalStats	Run totals kept by the main thread: totalReq, totalErrors, latencySum, successfulTicks, peakRps	Reset at the start of every test

Algorithm 1: Test Session Initialisation

This algorithm runs on the main thread when the dashboard sends the start request (POST /api/start-test). Its purpose is to guarantee that only one test is active at a time and that every run starts from clean statistics. Any worker left over from an earlier run is terminated, the global statistics are reset, the start time is recorded, and a new worker thread is created from the same source file with the test configuration passed as workerData.

```
ALGORITHM 1: Test Session Initialisation
INPUT  : targetUrl, virtualUsers, latency  (JSON body of the request)
OUTPUT : a running worker thread and an acknowledgement message
1.  FOR EACH w IN activeWorkers DO  w.terminate()
2.  activeWorkers <- empty list
3.  globalStats  <- {totalReq:0, totalErrors:0, latencySum:0,
                     successfulTicks:0, peakRps:0}
4.  testStartTime <- current time in milliseconds
5.  worker <- NEW Worker(server.js,
                workerData = {targetUrl, virtualUsers, latency})
6.  activeWorkers.push(worker)
7.  ON worker "message" EVENT (metrics) DO  run Algorithm 4
8.  RETURN {message: "Stress test initiated"}
```

Algorithm 2: Virtual-User Tick Generation with Chaos Injection

This is the central algorithm of SimuNet and it runs inside the worker thread. A timer fires once every 1000 ms. At each tick the worker creates N asynchronous tasks, where N is the current number of virtual users, and all tasks are started at the same moment. Each task records its start time, waits for the chaos latency L if L is greater than zero, sends an HTTP GET request to the target URL with Axios, and then classifies the outcome. A request that receives a successful response increments the success counter and adds its elapsed time to the total latency; a request that raises an error increments the error counter. When every task has finished, the worker sends one summary message to the main thread. Figure 5.1 shows the logic as a flowchart.

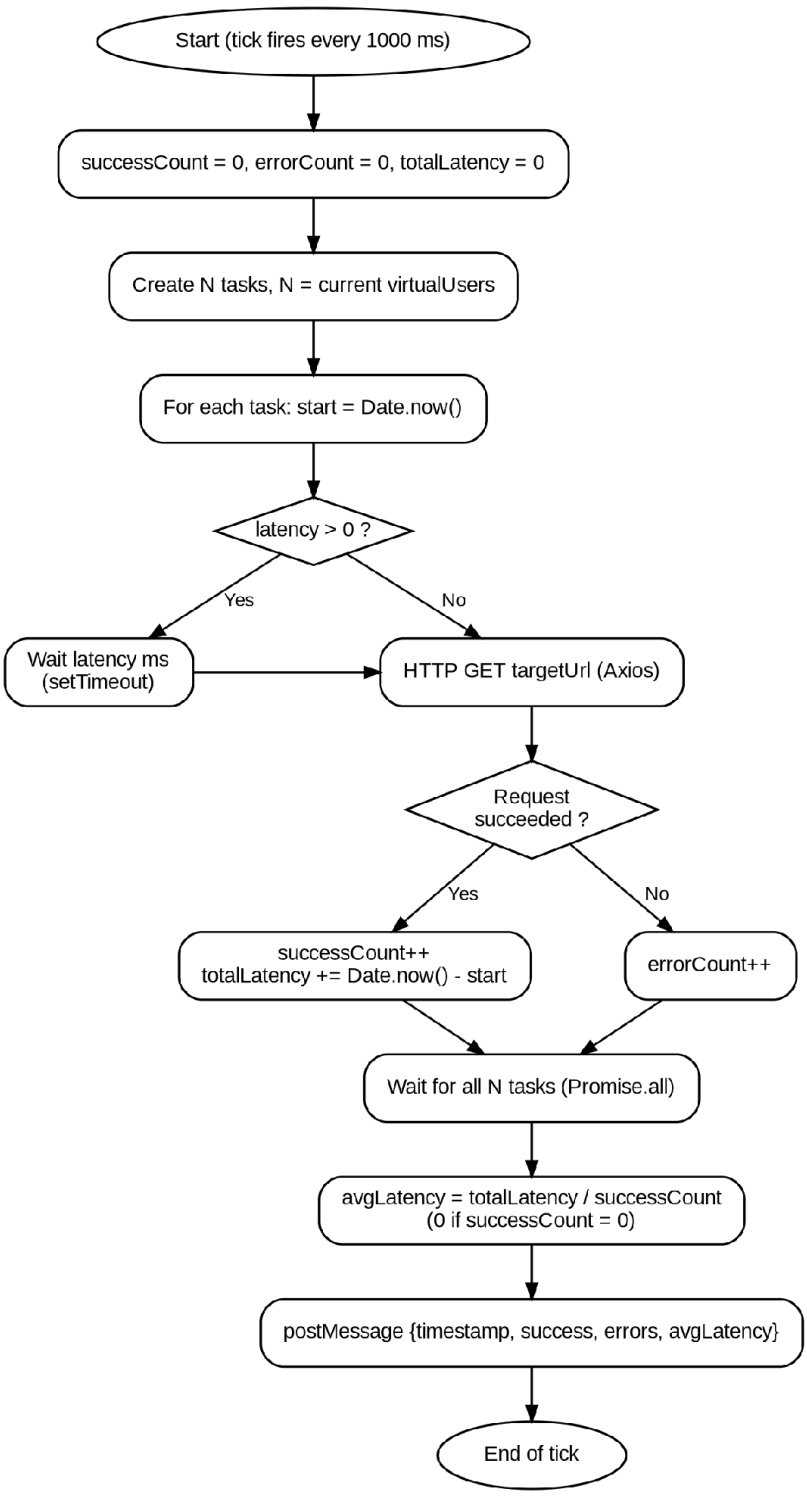
```
ALGORITHM 2: Virtual-User Tick Generation with Chaos Injection (worker thread)
INPUT  : workerData {targetUrl, virtualUsers, latency}, update messages
OUTPUT : one metrics message per tick
1.  EVERY 1000 ms DO  simulateTraffic()

simulateTraffic():
1.  success <- 0 ; errors <- 0 ; totalLatency <- 0
2.  FOR i <- 1 TO virtualUsers DO IN PARALLEL            // N async tasks
3.      start <- current time
4.      TRY
5.          IF latency > 0 THEN  wait(latency milliseconds)  // chaos delay
6.          HTTP GET targetUrl
7.          success <- success + 1
8.          totalLatency <- totalLatency + (current time - start)
9.      CATCH  errors <- errors + 1
10. AWAIT all N tasks
11. IF success > 0 THEN avgLatency <- totalLatency / success ELSE avgLatency <- 0
12. POST MESSAGE {timestamp, success, errors, avgLatency} TO main thread
```

Algorithm 3: Dynamic Parameter Update

SimuNet allows the number of virtual users and the chaos latency to be changed while a test is running, without stopping it. The sliders are not disabled during a run. On the front end, whenever either slider value changes while isRunning is true, the dashboard sends the new values to POST /api/update-params. The main thread forwards them to every active worker as a message, and the worker overwrites its local variables. Because the variables are read at the beginning of each tick, the new values take effect from the next tick onward.

Fig 5.3 flow chart



5.10 Case Study Design

A controlled case study can use the local `/api/ping` endpoint as the target. The endpoint responds with a small random delay and a JSON status value. The purpose is not to benchmark a production service, but to demonstrate the test workflow and observe how selected parameters affect the collected metrics. Keep the machine, browser, target, and run duration constant while varying one input at a time.

Experiment A: run with zero injected delay and a low load. Experiment B: keep the same load and increase artificial delay. Experiment C: keep the delay fixed and increase the load. For each experiment, record start time, duration, target URL, load parameter, artificial delay, total requests, errors, average latency, and peak interval count. Repeat each case at least three times if possible. If actual runs have not been performed, leave the result cells blank and label the table “planned test protocol” rather than inventing values.

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Development Environment and Setup

The project repository separates the frontend and backend into folders. The backend package defines `server.js` as the main server file and includes Express, Socket.IO, Axios, and CORS. The frontend package uses Vite scripts for development, build, linting, and preview. Node.js and npm must be installed before dependencies can be installed and the application started.

A typical local setup is: open a terminal in the backend directory and run `npm install`, then start the backend using `node server.js` or the configured development command. Open a second terminal in the frontend directory and run `npm install` followed by `npm run dev`. Open the local URL printed by Vite. Confirm that the backend reports that it is listening on port 4000 and that the browser can reach the backend routes. Replace these instructions if the final repository uses different commands.

6.2 Frontend Implementation Procedure

The frontend is built with React and Vite. The main implementation tasks are to define the dashboard layout, maintain input values for the target URL and load settings, call the backend control routes, establish a Socket.IO connection, and update visual components when new metrics arrive. Input validation should give visible feedback when values are missing or outside acceptable limits.

The frontend should disable or change the start/stop controls appropriately during a run. It should display connection state so that a user can distinguish a quiet test from a disconnected telemetry channel. A test that continues on the backend after the browser disconnects needs a defined policy: either stop automatically or continue under a bounded server-side session. The current code does not show a complete multi-session policy, so the final report should state that as a limitation.

6.3 Backend Implementation Procedure

The backend creates an Express application, enables CORS, parses JSON request bodies, creates an HTTP server, and attaches a Socket.IO server. The `/api/ping` route provides a simple JSON response after a short delay. The `/api/start-test` route resets counters and starts a worker thread. The `/api/update-params` route sends updated settings to active workers. The `/api/stop-test` route terminates workers and returns summary values.

The backend should be hardened before use beyond a local demonstration. CORS should be restricted to the actual frontend origin where possible. Request-body values should be validated and normalised. Target URLs should be restricted to approved hosts or an allowlist. A maximum duration and concurrency limit should be enforced on the server rather than trusting the browser controls. Error handling should return structured responses and log a reason without exposing sensitive internal details.

6.4 Worker Implementation Procedure

When `isMainThread` is true, the file starts the server. Otherwise, it runs the worker logic. The worker reads its initial configuration from `workerData` and listens for update messages on `parentPort`. The request simulation function initialises counters, creates asynchronous request operations, applies artificial delay when configured, issues Axios GET requests, and sends the resulting metrics to the parent thread.

The implementation is intentionally concise, but there are correctness issues worth reviewing. A periodic `setInterval` can start another simulation before a previous batch completes if the batch takes longer than the interval. The worker should use a non-overlapping loop or a guard that prevents overlapping batches. The implementation also needs a clear policy for request timeouts, aborted requests, status codes, and worker shutdown. These are concrete improvements that can be discussed honestly in the report.

6.5 Metric Definitions and Formulas

Total requests for an interval are calculated as successful requests plus errors reported by the worker. Total requests for the session are accumulated from these interval values. Error rate is calculated as total errors divided by total requests, multiplied by 100; if no requests occurred, the implementation returns zero to avoid division by zero. Average latency is calculated as the accumulated successful-request latency divided by the number of successful requests.

The current average latency excludes failed requests because latency is only accumulated for successful operations. A more complete performance report could include latency for failed requests and separate network errors from HTTP error responses. The peak interval metric is the largest `success + errors` count reported in a batch; because the batch is triggered by a timer and waits for completion, this value should be called peak interval request count unless the interval duration is measured and normalised accurately.

6.6 Testing Strategy

Testing should be organised into unit-level checks of metric formulas, integration checks of backend routes and worker messages, UI checks of start/stop and chart updates, and controlled load experiments against the local ping endpoint. Each test case should record preconditions, steps, expected behaviour, actual behaviour, and status. A test is “Pass” only after the behaviour is observed; otherwise mark it “Pending” or “Fail” rather than pre-filling a successful outcome.

A useful test matrix includes a valid start request, missing target URL, malformed URL, zero load, minimum permitted load, parameter update while running, successful target response, unreachable target, injected delay set to zero, injected delay set to a positive value, stop after a run, stop when idle, worker failure, browser disconnection, and repeated runs. Security checks should confirm that the service is not exposed as an unrestricted public load generator.

6.7 Test Cases

TC-01 — Local endpoint availability. Precondition: backend running. Action: open `/api/ping` using a browser or HTTP client. Expected: JSON response containing a status field. Actual result: [fill after test]. Status: [Pass/Fail/Pending].
TC-02 — Start test with valid parameters. Precondition: backend and frontend running and a permitted target available. Action: submit a valid target, load value, and delay. Expected: backend acknowledges the start and worker metrics begin arriving. Actual result: [fill after test]. Status: [Pass/Fail/Pending].
TC-03 — Inject artificial delay. Action: run the same target and load with zero delay, then repeat with a positive delay. Expected: the delay is applied before each request and measured latency generally rises; exact values depend on system and network conditions. Actual result: [fill after test]. Status: [Pass/Fail/Pending].
TC-04 — Stop test. Action: stop an active run. Expected: active workers are terminated and a final summary is returned. Actual result: [fill after test]. Status: [Pass/Fail/Pending].
TC-05 — Unreachable target. Action: use a permitted test URL that is deliberately unavailable. Expected: request failures are counted and the dashboard remains usable. Actual result: [fill after test]. Status: [Pass/Fail/Pending].
TC-06 — Dynamic update. Action: change the load or delay while the run is active. Expected: the backend forwards update messages and the worker uses the new values for subsequent batches.

6.8 Testing Results: How to Report Them

Do not fill in “Pass” merely because a test case has been written. Run the test and record the observed output. For each actual experiment, save a screenshot of the dashboard and, if possible, a log containing the configuration and summary. Include the date, system specifications, Node.js version, browser, target endpoint, run duration, and whether other applications were consuming resources.

The dashboard screenshot supplied with the project materials appears to show an example run with 18,260 total requests, 415 peak requests per second, 615.03 ms average latency, and 0.00% error rate under a displayed +400 ms chaos delay. These values should be included only as “values visible in the supplied demonstration screenshot” unless you can confirm the exact test setup and reproduce the run. They are not sufficient by themselves to validate engine accuracy or general system performance.

6.9 Implementation Limitations

The current server keeps global counters and a global worker list. This design is simple for a single-user demonstration but does not isolate simultaneous test sessions. A second start request terminates workers from the previous run and resets global counters. A multi-user deployment would need per-session state and access control.

The current code has limited URL and numeric validation, no explicit request timeout in the shown Axios call, and no visible maximum duration. Worker errors and exits are not comprehensively handled. The periodic worker loop may overlap if batches take longer than the timer period. Metrics are held in memory and are not durable after the process stops. The report should list these as limitations and proposed improvements rather than obscure them.

CHAPTER 7: ADVANTAGES AND DISADVANTAGES

7.1 Advantages

The first advantage is accessibility. A browser dashboard can present the main controls and metrics without requiring the user to learn a large test-plan format. The interface can make the test configuration and live behaviour visible in one place. The actual request generation remains in the backend rather than being performed by the browser itself.

The second advantage is modularity. The React frontend, Express control API, worker thread, HTTP client, and Socket.IO telemetry layer have distinct responsibilities. This separation makes it possible to improve one area without redesigning every part of the application. For example, the chart can be improved without changing the worker's request loop, provided the metric message schema remains stable.

The third advantage is educational value. The project demonstrates asynchronous requests, concurrency, HTTP communication, event-driven updates, metric aggregation, and configurable delay injection. It provides a base for experiments involving load, latency, and failure. The use of a local ping endpoint can reduce dependence on external APIs during initial testing.

The fourth advantage is that the core stack relies on widely used JavaScript tools. React and Vite support frontend development, Express provides routing, Axios issues HTTP requests, Node.js provides worker threads, and Socket.IO supports event-based communication. These components have official documentation and a broad developer community, which supports learning and maintenance.

7.2 Disadvantages

The project is a prototype rather than a mature performance-testing platform. It does not demonstrate distributed load generation across multiple machines, durable historical storage, a complete authentication system, comprehensive target protection, or a full experiment library. Its current global state is unsuitable for concurrent independent users without redesign.

The metric model is limited. Average latency does not show latency distribution, and the peak interval count is not necessarily a precisely measured per-second rate. Failed request latency is excluded from the current average. Without repeated controlled tests, the numbers may be affected by the test machine, target, network, and scheduling behaviour.

The request loop and lifecycle require additional hardening. The current implementation should add explicit input validation, request timeouts, concurrency limits, non-overlapping scheduling, worker error handling, and reliable cancellation. A load generator can itself become a source of resource exhaustion, so safe defaults and visible limits are important.

7.3 Comparison with Established Tools

SimuNet is not intended to prove that desktop or established testing tools are inferior. Mature tools often provide scripting, assertions, protocol support, distributed workers, reports, and integrations beyond the current project's scope. SimuNet instead focuses on a compact dashboard and a small set of metrics to help explain performance-testing concepts.

A fair comparison should evaluate the same target and workload, record each tool's configuration, and consider resource consumption, feature coverage, ease of setup, metric precision, repeatability, and safety controls. The report should not claim that SimuNet is faster or more accurate than JMeter or Postman without a controlled comparative experiment.

7.4 Ethical and Safe Use

A load test intentionally creates traffic and can degrade or interrupt a service. The operator must own the target or have explicit written permission to test it. Testing should begin at low concurrency, use bounded duration, and stop immediately if the target behaves unexpectedly. Public production endpoints should not be used for an academic demonstration without formal authorisation.

If SimuNet is deployed beyond localhost, access controls, target allowlists, request-rate limits, logging, and abuse monitoring should be added. CORS configuration alone is not authentication. A public service that accepts arbitrary target URLs can be misused to generate unwanted traffic. Responsible use is a design requirement, not merely a disclaimer in the report.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Results Overview

The supplied project materials show a dashboard designed to configure a target, choose load and artificial delay, view changing metrics, and stop a test to obtain a summary. The source code confirms a local ping endpoint and a backend flow for starting, updating, and stopping a worker. The presentation lists the dashboard UI, worker engine, real-time telemetry layer, and Chart.js visualisation as completed, while PDF export and multi-node scaling are marked pending.

The result section should contain screenshots from the actual running application. At minimum, include the initial dashboard, the dashboard during a run, the delay control, the live chart, the stop action, and the final summary. Captions should identify what each screenshot demonstrates. Avoid inserting a screenshot without explaining its relationship to a requirement or test case.

8.2 Interpreting the Metrics

Total requests indicate the number of requests accounted for by the worker messages during the run. A higher total does not automatically mean the target is better; it may mean that the run lasted longer or the test settings differed. The duration and configuration must be recorded alongside the total. For fair comparison, keep the run duration and target conditions consistent.

Average latency represents the mean elapsed time for successful requests in the current implementation. A higher average can result from injected delay, target processing time, queueing, network variation, or contention on the load generator. The metric does not identify the cause by itself. Use controlled experiments to isolate variables and repeat runs to assess variability.

Error rate is the percentage of counted errors among all counted requests. An error can reflect an unavailable target, network failure, timeout, or HTTP status behaviour as interpreted by Axios. The current implementation does not report a detailed breakdown of error causes. Future work should capture error categories and HTTP status codes without storing sensitive response data.

8.3 Demonstration Screenshot Values

The supplied dashboard screenshot appears to display the following values: 18,260 total requests, 415 peak requests per second, 615.03 ms average latency, 0.00% error rate, and +400 ms injected delay. These figures are transcribed from the image, not independently reproduced in this report. They should be labelled “illustrative values shown in the supplied screenshot” unless a test log confirms the exact configuration and run.

A zero error rate in one displayed run does not prove that the target is error-free under all loads. Similarly, a peak rate displayed by a prototype is not a benchmark unless the counting interval and timing are validated. The report should interpret the screenshot conservatively and avoid claims such as “the system can always handle 415 requests per second.”

8.4 Proposed Experiment Matrix

Experiment 1: baseline. Use the local ping endpoint, a low load value, zero artificial delay, and a fixed duration. Record the summary and capture the dashboard. Experiment 2: delay. Keep the target and load fixed but increase the artificial delay. Compare average latency and total completed requests. Experiment 3: load. Keep the target and delay fixed but increase the load setting in controlled increments. Monitor errors and system resource use.

Experiment 4: unavailable endpoint. Use a local endpoint that is intentionally stopped or a controlled invalid local port. Record whether the application counts failures and remains responsive. Experiment 5: dynamic update. Start at a low load, change the load value during the run, and observe whether subsequent metrics change. Experiment 6: stop behaviour. Stop a run during activity and verify that the worker terminates and a summary is returned. For every experiment, record actual observations and label any unexecuted test as pending.

8.5 Discussion

SimuNet demonstrates the architecture of a small load-testing workflow: configuration, request generation, metric collection, live event delivery, and summary calculation. The worker-thread model separates the request loop from the main server thread. Socket.IO provides an event-driven path for metrics, while the browser can render the data through chart components. These are appropriate choices for an educational proof of concept.

The main discussion point is the difference between a functional demonstration and a validated testing tool. A demonstration can show that a worker is created and metrics reach the dashboard. A validated tool must also establish correct counting, timing accuracy, repeatability, safe limits, failure handling, and session isolation. The project is a useful foundation, but additional engineering is required before using it for high-stakes capacity planning.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

SimuNet is a browser-oriented prototype for API load testing and basic delay injection. It combines a React/Vite frontend with an Express backend, a Node.js worker thread, Axios-based HTTP requests, Socket.IO metric delivery, and chart-based presentation. Its workflow enables a user to configure a target and load parameters, start a run, observe interval measurements, update parameters, and stop the run to obtain a summary.The project provides practical experience with web application architecture, asynchronous programming, worker lifecycle, telemetry, and measurement. The implementation and its limitations have been documented so that the report does not overstate what the current source code provides. In particular, persistent metrics, multi-node scaling, comprehensive fault injection, production security, and PDF export should be treated as future work unless they are implemented and verified in the final submission.

9.2 Limitations

The current prototype uses global in-memory state and is not designed for independent concurrent sessions. The input validation and safety controls are limited. The request loop requires protection against overlapping batches, and the backend should handle worker failures and shutdown consistently. Request timeouts and more detailed error categorisation should be added.

Metrics are basic and may not be sufficient for formal benchmarking. The current average latency is based on successful requests, and the peak interval count is not a fully validated rate measurement. The system does not persist historical runs, calculate latency percentiles, or demonstrate distributed load generation in the supplied code. These limitations restrict the conclusions that can be drawn from a run.

9.3 Future Enhancements

A first enhancement is input and safety hardening: validate URLs, restrict targets, cap concurrency and duration, configure request timeouts, add cancellation, and require authorisation for non-local targets. A second enhancement is session management: assign each run an identifier, store counters per session, and prevent one user's start request from terminating another user's test.

A third enhancement is metric quality. The worker can record request start and completion times using a monotonic clock, track latency for both successes and failures, and aggregate time buckets to calculate a more accurate throughput rate. Histograms or bounded sample storage can support p50, p90, p95, and p99 latency. Tests should verify each formula against known input data.

A fourth enhancement is persistence and reporting. A database or time-series store could retain run configurations and summary values. A report exporter could generate a PDF with test metadata, charts, a summary table, and a limitations statement. Redis, Docker, and multi-node execution could be introduced as future infrastructure after architecture and deployment requirements are specified. These should not be represented as existing capabilities until implemented.

Further extensions include additional fault modes, configurable timeouts, controlled failure percentages, CI/CD integration, test templates, comparative runs, accessibility improvements, and a safer target allowlist. Each enhancement should be developed incrementally and tested in an isolated environment. Distributed load generation should include coordination, clock considerations, per-node telemetry, and protection against one node becoming a bottleneck.

9.4 Applications

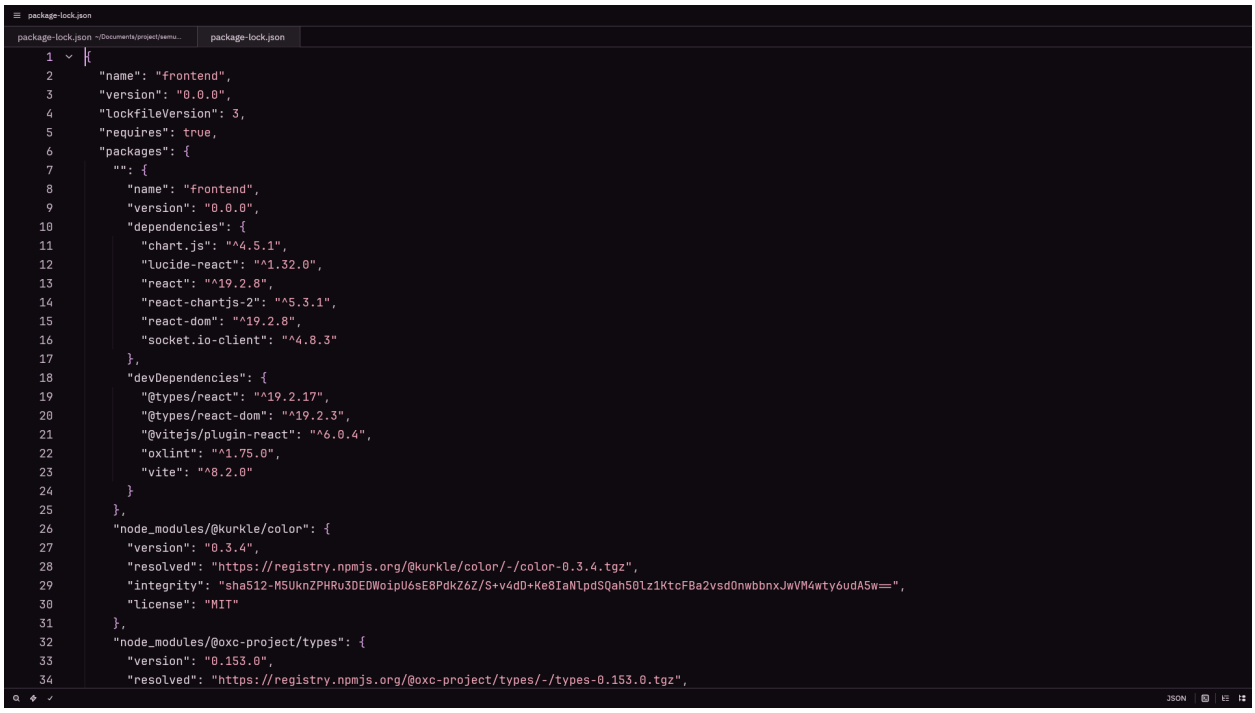
The system can be used in classroom demonstrations of API behaviour under concurrent requests. Students can compare low and high load settings and observe changes in the displayed metrics. It can also serve as a local development aid when a developer wants a simple way to exercise an API endpoint before integrating it with other components.

A controlled staging environment can be used to explore how added delay affects request completion, provided the operator has permission and the test is bounded. The prototype is not, by itself, a substitute for security testing, formal reliability testing, or a production-grade load-testing programme. Its most appropriate current use is educational and exploratory.

CHAPTER 10: APPENDICES

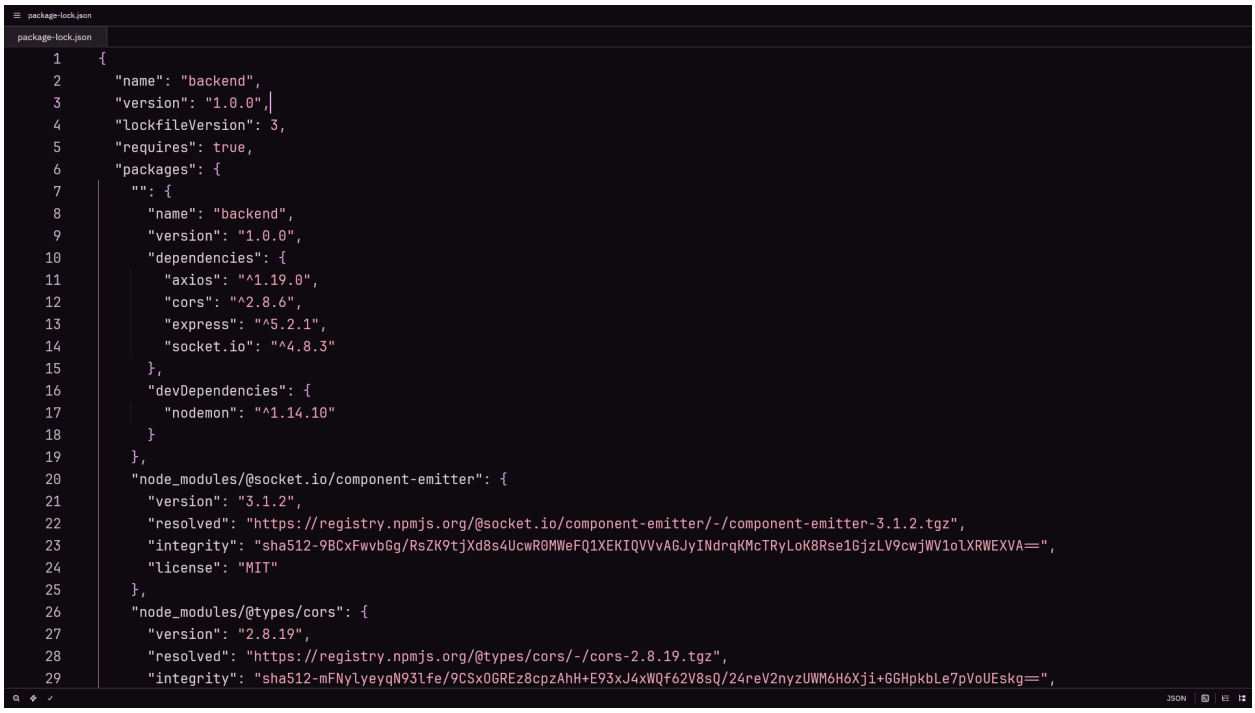
10.1 Sample code

Fig A.1 frontend



```
package-lock.json
package-lock.json ~/Documents/project/seemu...   package-lock.json
{
  "name": "frontend",
  "version": "0.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "frontend",
      "version": "0.0.0",
      "dependencies": {
        "chart.js": "^4.5.1",
        "lucide-react": "^1.32.0",
        "react": "^19.2.8",
        "react-chartjs-2": "^5.3.1",
        "react-dom": "^19.2.8",
        "socket.io-client": "^4.8.3"
      },
      "devDependencies": {
        "@types/react": "^19.2.17",
        "@types/react-dom": "^19.2.3",
        "@vitejs/plugin-react": "^6.0.4",
        "oxlint": "^1.75.0",
        "vite": "^8.2.0"
      }
    },
    "node_modules/@kurkle/color": {
      "version": "0.3.4",
      "resolved": "https://registry.npmjs.org/@kurkle/color/-/color-0.3.4.tgz",
      "integrity": "sha512-M5UknZPHRu3DEDWoipU6sE8PdkZ6Z/S+v4dD+Ke8IaNlpdSQah50lz1KtcFBa2vsdOnwbbnxJwVM4wty6udA5w==",
      "license": "MIT"
    },
    "node_modules/@oxc-project/types": {
      "version": "0.153.0",
      "resolved": "https://registry.npmjs.org/@oxc-project/types/-/types-0.153.0.tgz",
```

Fig A.2 backend



```
package-lock.json
package-lock.json
{
  "name": "backend",
  "version": "1.0.0",
  "lockfileVersion": 3,
  "requires": true,
  "packages": {
    "": {
      "name": "backend",
      "version": "1.0.0",
      "dependencies": {
        "axios": "^1.19.0",
        "cors": "^2.8.6",
        "express": "^5.2.1",
        "socket.io": "^4.8.3"
      },
      "devDependencies": {
        "nodemon": "^1.14.10"
      }
    },
    "node_modules/@socket.io/component-emitter": {
      "version": "3.1.2",
      "resolved": "https://registry.npmjs.org/@socket.io/component-emitter/-/component-emitter-3.1.2.tgz",
      "integrity": "sha512-9BCxFwvbGg/RsZK9tjXd8s4UcwR0MWeFQ1XEKIQVVvAGJyINdrqKMcTRyLoK8Rse1GjzLV9cwjWV1olXRWEXVA==",
      "license": "MIT"
    },
    "node_modules/@types/cors": {
      "version": "2.8.19",
      "resolved": "https://registry.npmjs.org/@types/cors/-/cors-2.8.19.tgz",
      "integrity": "sha512-mFNylyeyqN93lfe/9CSxOGREz8cpzAhH+E93xJ4xWQf62V8sQ/24reV2nyzUWM6H6Xji+GGHpkbLe7pVoUEskg==",
```

CHAPTER 11: REFERENCES

[1] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, “Chaos Engineering,” IEEE Software, vol. 33, no. 3, pp. 35–41, 2016, doi: 10.1109/MS.2016.60.

[2] J. Dean and L. A. Barroso, “The Tail at Scale,” Communications of the ACM, vol. 56, no. 2, pp. 74–80, 2013, doi: 10.1145/2408776.2408794.

[3] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Eds., Site Reliability Engineering: How Google Runs Production Systems. O’Reilly Media, 2016.

[4] M. T. Nygard, Release It!: Design and Deploy Production-Ready Software, 2nd ed. Pragmatic Bookshelf, 2018.

[5] Node.js Contributors, “Worker threads,” Node.js Documentation. Available: https://nodejs.org/api/worker_threads.html. Accessed: 12 Oct. 2026.

[6] Socket.IO, “Introduction,” Socket.IO Documentation. Available: https://socket.io/docs/v4/. Accessed: 12 Oct. 2026.

[7] Chart.js Contributors, “Getting Started,” Chart.js Documentation. Available: https://www.chartjs.org/docs/latest/getting-started/. Accessed: 12 Oct. 2026.

[8] R. Fielding, M. Nottingham, and J. Reschke, “HTTP Semantics,” RFC 9110, Internet Engineering Task Force, 2022. Available: https://www.rfc-editor.org/rfc/rfc9110.

[9] Node.js Contributors, “Node.js API documentation,” Node.js. Available: https://nodejs.org/api/. Accessed: 12 Oct. 2026.

[10] Express.js, “Express documentation,” Express. Available: https://expressjs.com/. Accessed: 12 Oct. 2026.

[11] Axios Contributors, “Axios documentation,” GitHub. Available: https://axios-http.com/docs/intro. Accessed: 12 Oct. 2026.

[12] React Team, “React documentation,” React. Available: https://react.dev/. Accessed: 12 Oct. 2026.

[13] Vite Contributors, “Vite guide,” Vite. Available: https://vite.dev/guide/. Accessed: 12 Oct. 2026.

[14] Apache Software Foundation, “Apache JMeter User’s Manual,” Apache JMeter. Available: https://jmeter.apache.org/usermanual/. Accessed: 12 Oct. 2026.

[15] Netflix, “Principles of Chaos Engineering,” Principles of Chaos Engineering. Available: https://principlesofchaos.org/. Accessed: 12 Oct. 2026.

[16] M. Fowler, “CircuitBreaker,” martinfowler.com, 2014. Available:

https://martinfowler.com/bliki/CircuitBreaker.html.

[17] J. Allspaw and P. Robbins, Web Operations: Keeping the Data on Time. O'Reilly Media, 2010.

[18] B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. Addison-Wesley, 2020.

[19] IEEE, IEEE Recommended Practice for Software Reliability, IEEE Std 1633-2016, 2016.

[20] IETF, "WebSocket Protocol," RFC 6455, 2011. Available: https://www.rfc-editor.org/rfc/rfc6455.

[21] Socket.IO, "Emitting events," Socket.IO Documentation. Available: https://socket.io/docs/v4/emitting-events/. Accessed: 12 Oct. 2026.

[22] Chart.js Contributors, "Line chart," Chart.js Documentation. Available: https://www.chartjs.org/docs/latest/charts/line.html. Accessed: 12 Oct. 2026.

[23] npm, "npm documentation," npm Docs. Available: https://docs.npmjs.com/. Accessed: 12 Oct. 2026.

[24] Express.js, "API reference," Express. Available: https://expressjs.com/en/5x/api.html. Accessed: 12 Oct. 2026.

[25] A. Cockcroft and D. Sheehan, "Benchmarking cloud performance," (Use only if your guide verifies a suitable exact publication record before submission; otherwise replace this entry with a verified paper approved by your guide).

turnitin Page 1 of 63 - Cover Page Submission ID trn:oid:::10159:154692400

stephin roll.no 51

SimuNet_final_report

Assignment @

Document Details

Submission ID
trn:oid:::10159:154692400

Submission Date
Oct 14, 2026, 9:21 PM GMT+5:30

Download Date
Oct 14, 2026, 9:45 PM GMT+5:30

File Name
SimuNet_final_report.docx

File Size
3.7 MB

60 Pages

12,455 Words

72,861 Characters

turnitin Page 1 of 63 - Cover Page Submission ID trn:oid:::10159:154692400



turnitin Page 2 of 63 - Integrity Overview Submission ID trn:oid:::10159:154692400

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text
- Small Matches (less than 18 words)

Match Groups

13 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

3% Internet sources
2% Publications
6% Submitted works (Student Papers)

turnitin Page 2 of 63 - Integrity Overview Submission ID trn:oid:::10159:154692400